

MSA기반 서비스 분해 전략 및 서비스 IPC 개발하기

마이크로서비스 소개

- 모놀리식 아키텍처
- 마이크로서비스 아키텍처
- 마이크로서비스 아키텍처의 장단점
- 모놀리식 아키텍처에서 마이크로서비스 아키텍처 전환
- 성공적인 마이크로서비스 아키텍처 구현을 위한 고려사항
- 마이크로서비스 아키텍처 패턴 언어
- 마이크로서비스 환경의 프로세스와 조직

모놀리식 아키텍처

- 모놀리식 아키텍처는 소프트웨어 개발에서 사용되는 한 가지 아키텍처 패턴으로, 애플리케이션을 단일한, 통합된 유닛으로 개발하는 방식을 말함.
- 특히 웹 서비스와 관련하여 널리 사용되며, 여러 구성 요소가 서로 밀접하게 연결되어 있고 동일한 프로세스에서 실행되는 구조를 가지고 있음.

모놀리식 아키텍처

- 모놀리식 아키텍처의 특징
 - 통합된 코드베이스: 모놀리식 애플리케이션은 단일 코드 저장소에 모든 애플리케이션 기능이 포함되어 있으며, 데이터베이스 액세스 로직, 비즈니스 로직, 그리고 UI 코드 등이 한데 모여 있음.
 - 단순한 배포: 전체 애플리케이션이 하나의 실행 가능한 파일이나 WAR 파일 등의 단일 아티팩트로 패키징되기 때문에 배포가 상대적으로 간단함.
 - 개발의 용이성: 개발자들은 애플리케이션의 다양한 부분을 쉽게 이해하고 수정할 수 있으므로, 새로운 기능의 추가나 기존 기능의 수정이 용이함.

모놀리식 아키텍처

- 모놀리식 아키텍처의 장점

- 간단한 개발 및 테스트: 하나의 애플리케이션으로 모든 것을 처리하기 때문에, 개발 및 테스트 환경 설정이 상대적으로 간단함.
- 간단한 배포: 애플리케이션을 한 번에 하나의 파일로 배포할 수 있으므로 배포 프로세스가 단순화됨.
- 기술 스택 통합: 동일한 기술 스택을 전체 애플리케이션에 걸쳐 사용할 수 있어 기술 선택의 복잡성이 줄어듦.

- 모놀리식 아키텍처의 단점

- 확장성 제한: 모놀리식 애플리케이션은 크기가 커짐에 따라 관리가 어려워지고, 특정 기능만을 확장하기가 어려울 수 있음.
- 기술 종속성: 한 번 선택한 기술 스택에 대한 종속성이 생겨, 새로운 기술로의 이동이 어려울 수 있음.
- 개발 속도 저하: 애플리케이션의 크기가 커지면, 빌드 시간이 길어지고, 개발 및 배포 프로세스가 느려질 수 있음.

모놀리식 아키텍처

- 모놀리식 아키텍처는 특히 초기 단계의 프로젝트나 규모가 작고 복잡성이 낮은 애플리케이션에 적합할 수 있음.
- 애플리케이션이 성장하고 요구 사항이 복잡해지면서 확장성이나 유지 보수성의 문제로 인해 마이크로서비스 아키텍처로 전환을 고려하는 경우가 많음. 시스템의 확장성 유지보수성 및 개발 효율성을 향상시킬 수 있기 때문임.

마이크로서비스 아키텍처

- 마이크로서비스 아키텍처는 애플리케이션을 작고 독립적으로 배포 가능한 서비스들의 집합으로 구성하는 설계 방식임.
- 각 마이크로서비스는 특정 비즈니스 기능을 수행하고, 서비스 간에는 네트워크를 통해 통신함.
- 모놀리식 접근 방식과 대비되며, 애플리케이션의 확장성과 유지 보수성을 향상시키기 위해 고안되었음.

마이크로서비스 아키텍처

- 마이크로서비스 아키텍처의 주요 특징
 - 분산된 서비스: 애플리케이션은 서로 독립적으로 실행되는 여러 서비스로 나뉘며, 각 서비스는 특정 기능을 담당함.
 - 독립적 배포: 각 마이크로서비스는 독립적으로 배포될 수 있어, 특정 서비스의 업데이트가 다른 서비스에 영향을 미치지 않음.
 - 기술 다양성: 서비스별로 가장 적합한 기술 스택을 자유롭게 선택할 수 있어, 다양한 기술과 프로그래밍 언어를 함께 사용할 수 있음.
 - 분산 데이터 관리: 각 서비스는 자체 데이터베이스를 가질 수 있으며, 데이터 관리 방식이 서비스별로 다를 수 있음.
 - 유연한 확장성: 특정 서비스만을 대상으로 확장할 수 있어, 리소스 사용을 최적화하고 애플리케이션의 전반적인 성능을 향상시킬 수 있음.

마이크로서비스 아키텍처의 장단점

- 마이크로서비스 아키텍처의 장점

- 확장성: 비즈니스 요구에 따라 특정 서비스를 독립적으로 확장할 수 있음.
- 유연성: 서비스별로 다른 기술을 사용할 수 있어, 새로운 기술 도입이 용이함.
- 복원력: 하나의 서비스에 문제가 발생해도 전체 시스템에 영향을 미치지 않으므로, 시스템의 전반적인 안정성이 향상됨.
- 빠른 시장 출시: 서비스를 독립적으로 개발하고 배포할 수 있어, 새로운 기능의 추가나 변경이 빠르게 이루어질 수 있음.

마이크로서비스 아키텍처의 장단점

- 마이크로서비스 아키텍처의 단점
 - 복잡성 증가: 서비스 간의 통신, 데이터 일관성 유지, 서비스 디스커버리 등의 관리가 필요함.
 - 운영의 복잡성: 독립적으로 배포되고 운영되는 여러 서비스를 관리하려면, 자동화된 도구와 프로세스가 필요함.
 - 네트워크 지연: 서비스 간 통신은 네트워크를 통해 이루어지므로, 지연 시간이 발생할 수 있음.
 - 데이터 일관성 관리: 각 서비스가 독립된 데이터베이스를 사용할 경우, 전체 시스템에서 데이터 일관성을 유지하기 위한 추가 작업이 필요함.
- 마이크로서비스 아키텍처는 대규모, 복잡한 애플리케이션의 개발과 유지 보수에 특히 유리함.
- 기업이 빠르게 변화하는 시장 요구와 기술 발전에 유연하게 대응할 수 있도록 도와주며, 각 서비스를 독립적으로 확장하고 업데이트할 수 있어 전체 시스템의 안정성과 지속 가능성을 높임.

모놀리식 아키텍처에서 마이크로서비스 아키텍처 전환

- 많은 조직들이 시스템의 확장성과 유연성을 개선하기 위해 모놀리식 아키텍처에서 마이크로서비스 아키텍처로 전환을 고려하고 있음. 이 전환 과정에서는 다음과 같은 사항들을 고려해야 함.
 - 점진적 분해: 전체 시스템을 한 번에 마이크로서비스로 분해하는 것은 매우 복잡하고 리스크가 높음. 대신, 가장 중요하거나 변경이 잦은 기능을 먼저 분해하는 등 점진적으로 접근하는 것이 권장됨.
 - 도메인 주도 설계(DDD): 마이크로서비스 아키텍처 설계에 있어 도메인 주도 설계를 적용하여 각 서비스의 경계를 명확히 하는 것이 중요함. 이를 통해 각 마이크로서비스가 독립적인 비즈니스 기능을 담당하도록 함.
 - 통신 및 데이터 관리 전략: 마이크로서비스 간의 통신 방식과 데이터 관리 전략을 명확히 해야 합니다. 각 서비스는 독립된 데이터 소스를 가질 수 있으며, 서비스 간 통신은 REST API, gRPC 등을 통해 이루어질 수 있음.
 - 테스트 및 배포 전략: 마이크로서비스 각각을 독립적으로 테스트하고 배포할 수 있는 전략이 필요함. CI/CD 파이프라인을 구축하여 자동화된 테스트와 배포 과정을 적용하는 것이 효과적임.

모놀리식 아키텍처에서 마이크로서비스 아키텍처 전환

- 모놀리식 아키텍처는 그 구조의 단순함과 개발 초기 단계에서의 효율성 때문에 여전히 매력적인 선택지가 될 수 있음.
- 어플리케이션의 규모가 커지고 비즈니스 요구사항이 복잡해짐에 따라, 마이크로서비스 아키텍처로의 전환을 고려하는 것이 많은 이점을 제공할 수 있음.
- 중요한 것은 어플리케이션의 요구사항, 팀의 역량, 그리고 비즈니스 목표에 가장 적합한 아키텍처 스타일을 선택하는 것임.

성공적인 마이크로서비스 아키텍처 구현을 위한 고려사항

- 서비스 분할: 각 서비스는 단일 책임 원칙을 따라야 하며, 분명한 비즈니스 기능을 수행해야 함. 서비스 간의 의존성은 최소화하면서도, 각 서비스가 독립적으로 유용한 기능을 제공할 수 있어야 함.
- 통신 메커니즘: 서비스 간 통신을 위한 효율적이고 안정적인 메커니즘을 구축해야 함. RESTful API, gRPC, 메시지 큐 등 다양한 통신 방식 중 애플리케이션 요구에 맞는 방식을 선택할 수 있음.
- 서비스 디스커버리: 마이크로서비스 환경에서는 서비스가 동적으로 생성되고 소멸할 수 있으므로, 서비스 디스커버리 메커니즘을 통해 서비스 간의 위치를 자동으로 파악하고 연결할 수 있어야 함.
- 데이터 관리: 데이터 일관성과 트랜잭션 관리는 마이크로서비스 아키텍처의 주요 도전 과제 중 하나임. 이벤트 소싱, CQRS(명령 쿼리 책임 분리) 같은 패턴을 통해 데이터 일관성 문제를 해결할 수 있음.
- 보안: 서비스 간의 통신은 보안이 중요하므로, 적절한 인증과 권한 부여 메커니즘을 구현해야 함. API 게이트웨이를 사용하여 외부 요청을 관리하고, 서비스 간 통신을 보호할 수 있음.

성공적인 마이크로서비스 아키텍처 구현을 위한 고려사항

- 모니터링과 로깅: 분산 시스템에서는 문제 발생 시 신속한 진단과 해결이 중요함. 중앙 집중식 로깅, 모니터링 도구를 사용하여 시스템 전반에 걸쳐 상태를 추적하고 문제를 식별할 수 있어야 함.
- 마이크로서비스 아키텍처는 이러한 고려사항을 충분히 고려하고 적절히 설계할 때, 기업의 기술적 민첩성을 높이고, 복잡한 시스템의 개발 및 운영에 있어 많은 이점을 제공함.
- 모든 상황에 마이크로서비스가 적합한 것은 아니며, 도입 전 비즈니스 요구사항, 팀의 역량, 기존 시스템의 복잡성 등을 충분히 고려해야 함.

마이크로서비스 아키텍처 패턴 언어

- 마이크로서비스 아키텍처 패턴 언어(Microservices Architecture Pattern Language)는 마이크로서비스를 설계, 개발, 배포, 운영하기 위해 사용되는 일련의 패턴들을 체계적으로 정리한 것임.
- 패턴 언어는 개발자와 아키텍트가 마이크로서비스 기반 시스템을 구축할 때 직면하는 공통적인 문제들에 대한 해결책을 제공함.
- 각 패턴은 특정 문제를 해결하는 방법을 설명하며, 마이크로서비스 아키텍처의 설계와 구현에 있어서 지침을 제공함.

마이크로서비스 아키텍처 패턴 언어

- 마이크로서비스 아키텍처 패턴 언어의 중요성
 - 마이크로서비스 아키텍처 패턴 언어는 개발자와 아키텍트가 마이크로서비스 기반 시스템을 설계할 때 직면하는 복잡한 문제들에 대한 검증된 해결책을 제공함.
 - 패턴들은 단순한 지침서 이상의 역할을 하며, 성공적인 마이크로서비스 아키텍처 구축을 위한 필수적인 요소임.
 - 각 패턴은 특정 문제에 대한 해결책을 제시하며, 이를 통해 시스템의 전반적인 품질을 개선하고, 유지보수를 용이하게 하며, 비즈니스 가치를 극대화할 수 있음.
 - 적절한 패턴의 선택과 적용은 프로젝트의 성공에 결정적인 요소가 될 수 있으며, 마이크로서비스 아키텍처를 구축하는 과정에서 발생할 수 있는 다양한 도전과제를 극복하는 데 중요한 역할을 함.
 - 마이크로서비스 아키텍처 패턴 언어는 현대적인 소프트웨어 개발 환경에서 매우 중요한 자산이며, 지속적으로 발전하고 새로운 패턴이 추가될 것으로 예상됨.

마이크로서비스 아키텍처 패턴 언어

- 주요 마이크로서비스 아키텍처 패턴
 - Decomposition Patterns (분해 패턴)
 - Decompose by Business Capability (비즈니스 능력에 따른 분해): 시스템을 비즈니스 기능에 따라 독립적인 서비스로 분해함.
 - Decompose by Subdomain (서브도메인에 따른 분해): 도메인 주도 설계(DDD)의 개념을 활용하여 서브도메인 별로 서비스를 분리함.
 - Integration Patterns (통합 패턴)
 - API Gateway Pattern (API 게이트웨이 패턴): 모든 클라이언트 요청을 단일 진입점을 통해 서비스로 라우팅하는 구조를 만듦.
 - Event-Driven Data Management (이벤트 주도 데이터 관리): 서비스 간의 결합도를 낮추기 위해 이벤트를 사용하여 데이터를 비동기적으로 동기화함.

마이크로서비스 아키텍처 패턴 언어

- Database Patterns (데이터베이스 패턴)
 - Database per Service (서비스별 데이터베이스): 각 마이크로서비스가 자체 데이터베이스를 가지고, 데이터베이스를 서비스와 함께 독립적으로 관리함.
 - Shared Database (공유 데이터베이스): 여러 서비스가 같은 데이터베이스를 공유하지만, 복잡성과 결합도 문제를 야기할 수 있음.
- Deployment Patterns (배포 패턴)
 - Single Service per Host (호스트당 단일 서비스): 각 마이크로서비스를 독립적인 호스트 또는 VM에 배포함.
 - Serverless Deployment (서버리스 배포): 서비스를 서버리스 플랫폼에 배포하여 인프라 관리 부담을 줄임.

마이크로서비스 아키텍처 패턴 언어

- Observability Patterns (관찰 가능성 패턴)
 - Log Aggregation (로그 집계): 모든 서비스의 로그를 중앙 위치에서 수집하고 분석함.
 - Distributed Tracing (분산 추적): 서비스 간 호출을 추적하여 시스템의 동작을 이해하고 문제를 진단함.
- Reliability Patterns (신뢰성 패턴)
 - Circuit Breaker (서킷 브레이커): 장애가 발생한 서비스 호출을 중단하여 시스템 전체의 장애 확산을 방지함.
 - Retry Patterns (재시도 패턴): 일시적인 실패에 대응하여 요청을 자동으로 재시도함.
 - Timeouts (타임아웃): 네트워크 호출에 타임아웃을 설정하여, 시스템이 무한히 대기 상태에 빠지지 않도록 함.

마이크로서비스 아키텍처 패턴 언어

- Security Patterns (보안 패턴)
 - Identity and Access Management (IAM): 사용자의 신원을 확인하고 적절한 자원에 대한 접근을 관리함.
 - API Security (API 보안): SSL, OAuth, API 키와 같은 메커니즘을 통해 API를 보호함.
- Scalability Patterns (확장성 패턴)
 - Autoscaling (자동 스케일링): 로드에 따라 서비스 인스턴스의 수를 자동으로 조정함.
 - Back Pressure (백 프레셔): 시스템의 과부하 상태를 방지하기 위해, 요청을 조절하는 메커니즘을 구현함.

마이크로서비스 아키텍처 패턴 언어

- 패턴들은 마이크로서비스 아키텍처를 설계하고 구현하는 데 있어 필수적인 지침을 제공함.
- 각 패턴은 특정 문제를 해결하는 동시에, 마이크로서비스 아키텍처의 전반적인 성능, 확장성, 유연성, 신뢰성을 향상시키는 데 기여함.
- 패턴들을 적절히 조합하고 적용함으로써, 복잡한 시스템도 효과적으로 관리하고, 빠르게 변화하는 비즈니스 요구사항에 유연하게 대응할 수 있음.

마이크로서비스 환경의 프로세스와 조직

- 마이크로서비스 환경에서 프로세스와 조직 구조는 전통적인 모놀리식 개발 방식과 크게 다름.
- 이러한 환경은 더 민첩하고, 독립적인 팀의 협업을 촉진하며, 빠른 변화와 혁신을 지원하는 방식으로 설계되어야 함.
- 마이크로서비스 아키텍처를 채택함으로써 조직은 기술적으로는 물론 문화적으로도 변화를 겪게 됨.

마이크로서비스 환경의 프로세스와 조직

- 조직 구조 변화
 - DevOps 문화의 채택
 - 마이크로서비스 환경에서는 개발(Dev)과 운영(Ops) 팀 간의 경계가 모호해지며, 이 두 영역이 긴밀하게 협력하는 DevOps 문화가 필수적임. 이러한 문화는 지속적인 통합(CI), 지속적인 배포(CD), 자동화된 테스트, 모니터링 등을 포함함.
- 소규모, 자율적 팀
 - 마이크로서비스는 소규모 팀이 전체 서비스의 생명 주기를 관리할 수 있도록 함. 각 팀은 특정 비즈니스 기능이나 서비스에 대한 전체 책임을 지며, 설계부터 개발, 배포, 운영에 이르기까지 모든 과정을 담당함. 이를 통해 의사결정 속도가 빨라지고 유연성이 증가함.

마이크로서비스 환경의 프로세스와 조직

- 크로스 펑셔널 팀
 - 각 마이크로서비스 팀은 개발자, QA 전문가, 운영 전문가 등 다양한 역할의 사람들로 구성됨. 이는 팀 내에서 필요한 모든 역량을 갖추고, 서비스를 종단간(end-to-end)으로 관리할 수 있게 함.

마이크로서비스 환경의 프로세스와 조직

- 프로세스 변화
 - 지속적인 통합과 지속적인 배포 (CI/CD)
 - 마이크로서비스 아키텍처는 작은 단위로 자주 배포하는 것을 장려함. CI/CD 파이프라인을 통해 코드 변경 사항이 자동으로 테스트되고, 프로덕션 환경으로 빠르게 배포될 수 있음.
- 독립적인 서비스 배포
 - 각 마이크로서비스는 독립적으로 배포될 수 있어야 함. 이는 서비스별로 다른 배포 스케줄을 가능하게 하며, 전체 시스템의 안정성에 영향을 주지 않고 개별 서비스를 업데이트할 수 있음.

마이크로서비스 환경의 프로세스와 조직

- 분산된 거버넌스
 - 마이크로서비스 아키텍처에서는 중앙집중식 거버넌스보다는 분산된 거버넌스가 효과적임. 각 팀이 자신의 서비스에 대한 결정을 내릴 수 있는 권한을 갖고, 공통된 가이드라인 내에서 자율적으로 움직일 수 있어야 함.
- 실패를 용인하는 문화
 - 분산 시스템에서는 실패가 불가피함. 마이크로서비스 환경에서는 이러한 실패를 빠르게 감지하고 대응할 수 있는 문화와 시스템이 필요함. 이를 위해 실패를 용인하고, 실패로부터 배우며, 시스템의 복원력을 향상시키는 방향으로 조직 문화를 구축해야 합니다. 실패를 실험과 학습의 기회로 보는 것은 마이크로서비스 아키텍처를 성공적으로 운영하는 데 중요한 요소임.

마이크로서비스 환경의 프로세스와 조직

- 조직 문화의 변화
 - 팀 간의 협업 강화
 - 마이크로서비스 아키텍처는 팀 간의 경계를 모호하게 만들고, 상호 의존성을 최소화하면서도, 서비스 간의 통합을 위한 긴밀한 협업을 요구함. 이를 위해 커뮤니케이션과 협업 도구의 사용, 공통된 목표를 향한 팀워크의 중요성이 강조됨.
- 지속적인 학습과 혁신
 - 기술과 시장의 빠른 변화에 대응하기 위해, 마이크로서비스를 도입한 조직은 지속적인 학습과 혁신을 문화로 삼아야 함. 새로운 기술, 도구, 프로세스에 대한 교육과 실험을 장려하며, 직원들이 끊임없이 성장하고 발전할 수 있는 환경을 제공해야 함.

마이크로서비스 환경의 프로세스와 조직

- 마이크로서비스 환경의 성공적인 도입과 운영은 단순히 기술적인 변화를 넘어서 조직 문화와 프로세스의 근본적인 변화를 요구함.
- 변화를 통해 조직은 더 민첩하고, 반응이 빠르며, 시장 변화에 유연하게 대응할 수 있는 체계를 갖출 수 있음.
- 각 팀의 독립성과 자율성을 보장하면서도 전체 조직의 목표와 일관성을 유지하는 것이 중요함.
- 조직 내에서의 투명한 커뮤니케이션, 명확한 목표 설정, 그리고 공유된 비전에 대한 강한 커밋먼트를 통해 달성될 수 있음.

마이크로서비스 환경의 프로세스와 조직

- 조직 변화 관리

- 마이크로서비스 아키텍처로의 전환 과정에서 조직 변화 관리는 매우 중요한 역할을 합니다. 기존의 작업 방식과 문화에서 벗어나 새로운 시스템을 받아들이려면, 조직 전체의 마인드셋이 변화해야 합니다. 이를 위한 몇 가지 접근 방법은 다음과 같습니다.
- 리더십의 역할: 변화를 주도하는 리더들이 명확한 비전을 제시하고, 변화의 필요성을 소통해야 합니다. 리더들은 변화 과정에서 발생할 수 있는 불확실성과 저항을 관리하며, 팀을 지원해야 합니다.
- 교육과 훈련: 기술적 스킬 뿐만 아니라 새로운 작업 방식과 문화에 대한 교육과 훈련이 필요합니다. 지속적인 학습 기회를 제공하여 직원들이 새로운 환경에 적응할 수 있도록 해야 합니다.
- 소통과 피드백: 변화 과정에서 지속적인 소통이 중요합니다. 변화에 대한 직원들의 의견을 수렴하고, 이를 반영하여 과정을 조정할 수 있어야 합니다.
- 소규모에서 시작하기: 전 조직적인 변화보다는 소규모의 프로젝트나 팀부터 시작하여 점차 확대하는 것이 효과적일 수 있습니다. 이를 통해 초기 단계에서의 성공 사례를 기반으로 조직 내 확산을 시도할 수 있습니다.

마이크로서비스 환경의 프로세스와 조직

- 마이크로서비스 아키텍처는 단순한 기술적 선택 이상의 의미를 가지며, 조직 문화와 구조에 깊은 영향을 미침.
- 변화를 성공적으로 관리하기 위해서는 전략적 계획, 지속적인 지원, 그리고 조직 전반에 걸친 커밋먼트가 필요함.
- 마이크로서비스 도입은 단기간에 이루어지는 변화가 아니며, 지속적인 개선과 학습을 통해 점진적으로 발전시켜 나가야 하는 과정임.
- 조직은 기술적 도전과제를 극복하고, 변화하는 비즈니스 환경에 더 잘 대응할 수 있는 유연하고 민첩한 체계를 구축할 수 있음.

개발 환경 구성 및 설정

- Windows 10 pro
- VirtualBox or Hyper-V
- Install Ubuntu on VirtualBox or Hyper-V
- Docker Desktop
- Download Sources
- Build Sources
- Test on Containers
- <https://github.com/Dennis-IDEACUBE/KUT-MSA-Design-Pattern-1>

분해 전략

- 마이크로서비스 아키텍처란 정확히 무엇인가?
- 어플리케이션의 마이크로서비스 아키텍처 정의

마이크로서비스 아키텍처란 정확히 무엇인가?

- 소프트웨어 아키텍처의 정의
- 소프트웨어 아키텍처 스타일
- 마이크로서비스 아키텍처는 아키텍처 스타일

소프트웨어 아키텍처의 정의

- "A Definition of Software Architecture"는 소프트웨어 아키텍처에 대한 정의를 제공하는 문서나 개념을 가리킴.
- 이 용어는 소프트웨어 엔지니어링 분야에서 소프트웨어의 고수준 구조를 설명하고, 그 구조가 시스템의 기능과 품질 속성에 어떻게 영향을 미치는지를 명확히 하는 데 사용됨.
- 소프트웨어 아키텍처의 정의는 다양한 관점과 필요에 따라 여러 형태로 제시될 수 있지만, 일반적으로 다음과 같은 요소들을 포함함
 - 컴포넌트: 시스템을 구성하는 기본 단위로, 소프트웨어의 각 부분을 의미함.
 - 연결: 컴포넌트 간의 관계를 설명하며, 이들이 어떻게 데이터와 제어 흐름을 주고받는지를 명시함.
 - 제약사항: 시스템 설계와 구현에 영향을 미치는 기술적, 운영적 제한사항임.
 - 기술적 선택: 특정 기술 스택, 프로그래밍 언어, 미들웨어 등 시스템을 구축하는 데 사용되는 기술적 요소들임.

소프트웨어 아키텍처의 정의

- 소프트웨어 아키텍처의 정의는 시스템의 설계 초기 단계에서 중요한 역할을 하며, 개발 프로세스 전반에 걸쳐 시스템의 구조를 지침으로 사용됨.
- 프로젝트 이해관계자 간의 의사소통 도구로서의 역할도 수행하며, 시스템의 기술적 비전을 명확하게 하고, 복잡성을 관리하며, 프로젝트 목표 달성을 위한 공통의 이해를 구축하는 데 기여함.
- 소프트웨어 아키텍처를 효과적으로 정의하고 구현하는 것은 시스템의 성공적인 개발과 운영에 결정적인 요소로, 시스템의 확장성, 유지보수성, 성능, 보안 등 다양한 품질 속성에 직접적인 영향을 미침.

소프트웨어 아키텍처의 정의

- "4+1 뷰 모델(View Model) of Software Architecture"은 필립 크루치튼(Philippe Kruchten)이 제안한 소프트웨어 아키텍처를 설명하기 위한 프레임워크임.
- 이 모델은 시스템 아키텍처를 다양한 관점에서 바라보고 이해하기 위해 다섯 가지의 관점(뷰)을 사용함.
- 각각의 뷰는 시스템의 다른 측면을 다루며, 특정 이해관계자 그룹의 요구사항을 충족시키기 위해 설계되었음.
- "4+1"이라는 이름은 네 가지 주요 뷰와 이를 통합하는 하나의 시나리오 뷰를 의미함.

소프트웨어 아키텍처의 정의

- 논리적 뷰 (Logical View)
 - 시스템의 기능성을 설명함.
 - 주로 시스템을 사용하는 사용자(개발자를 포함)의 관점을 반영함.
 - 시스템의 주요 클래스, 패키지, 컴포넌트를 통해 소프트웨어의 구조를 보여줌.
- 개발 뷰 (Development View)
 - 시스템의 소프트웨어 아키텍처를 구현 관점에서 설명함.
 - 소프트웨어의 소스 코드 관리, 빌드 시스템, 모듈화 등 개발자의 관점을 반영함.
 - 소프트웨어 모듈의 조직, 소스 트리 구조, 각 모듈의 책임 등을 다룸.

소프트웨어 아키텍처의 정의

- 프로세스 뷰 (Process View)

- 시스템의 동적인 측면을 다루며, 런타임 시의 프로세스 간 상호작용을 설명함.
- 시스템의 성능, 동시성, 분산 시스템의 특성 등을 다룸.
- 시스템의 주요 프로세스, 스레드, 그리고 이들 간의 통신 방법을 포함함.

- 물리적 뷰 (Physical View)

- 시스템의 소프트웨어가 어떻게 하드웨어에 배치되는지를 설명함.
- 네트워크, 컴퓨터, 스토리지와 같은 하드웨어 자원의 배치와 시스템의 물리적 토폴로지를 다룸.
- 시스템의 확장성, 가용성, 장애 복구 등을 다루며, 시스템 운영자의 관점을 반영함.

소프트웨어 아키텍처의 정의

- +1. 시나리오 뷰 (Scenarios View)
 - 네 가지 뷰를 통합하며, 시스템의 다양한 사용 사례를 시나리오 또는 유스 케이스 형태로 설명함.
 - 시스템의 행동을 설명하며, 각 뷰가 서로 어떻게 연관되어 있는지를 보여줌.
 - 아키텍처의 검증과 이해관계자 간의 커뮤니케이션 도구로 사용됨.

소프트웨어 아키텍처의 정의

- 특히, 시나리오 뷰는 네 가지 주요 뷰를 통합하는 역할을 하면서도, 시스템이 실제로 사용되는 상황에서의 요구사항과 제약조건을 실질적으로 검증하는 메커니즘을 제공함. 이를 통해 아키텍처의 타당성을 검토하고, 필요한 조정을 가이드 할 수 있음.
- 아키텍처의 "4+1 뷰 모델"은 소프트웨어 개발 프로젝트에서 복잡한 시스템의 설계와 구현을 위한 강력한 도구로 활용될 수 있음.
- 각 뷰는 시스템을 다루는 특정한 관점을 제공하며, 시나리오를 통해 이러한 관점들이 실제 요구사항과 어떻게 일치하는지를 확인할 수 있음.
- "4+1 뷰 모델"은 아키텍트와 개발팀이 시스템의 주요 측면을 종합적으로 이해하고, 이해관계자들과 효과적으로 소통하며, 시스템의 품질과 성공적인 구현을 보장하는 데 중요한 역할을 함.

소프트웨어 아키텍처의 정의

- 아키텍처가 중요한 이유
 - 시스템의 기초 설정: 아키텍처는 소프트웨어 개발 프로젝트의 기반을 마련합니다. 초기 설계 단계에서 결정된 아키텍처는 개발 과정의 방향을 정하고, 품질, 성능, 확장성 등의 핵심 요소를 결정함.
 - 품질 보장: 아키텍처는 시스템의 비기능적 요구사항(성능, 보안, 유지보수성 등)을 충족하는 데 핵심적인 역할을 함. 잘 설계된 아키텍처는 이러한 품질 속성을 개선하고, 시스템이 사용자와 비즈니스의 기대를 만족시킬 수 있도록 도움.
 - 복잡성 관리: 복잡한 시스템을 효과적으로 관리하고 이해하기 위해서는 체계적인 아키텍처가 필요함. 아키텍처는 시스템의 다양한 부분을 조직화하고, 각 컴포넌트 간의 관계를 명확하게 정의함으로써, 전체 시스템을 더 쉽게 이해하고 관리할 수 있게 함.
 - 변경 용이성 및 확장성: 시장과 기술의 변화에 따라 소프트웨어 시스템은 지속적으로 변화하고 발전해야 함. 잘 정의된 아키텍처는 시스템의 변경과 확장을 용이하게 하며, 새로운 요구사항이나 기술을 효율적으로 통합할 수 있는 기반을 제공함.

소프트웨어 아키텍처의 정의

- 재사용성 증가: 아키텍처는 재사용 가능한 컴포넌트와 패턴을 정의함으로써, 개발 시간과 비용을 절약할 수 있게 합니다. 이를 통해 개발 효율성이 향상되며, 일관된 품질을 유지할 수 있음.
- 이해관계자 간의 의사소통: 아키텍처는 개발자, 관리자, 고객 등 프로젝트의 다양한 이해관계자들 사이의 공통 언어 역할을 함. 아키텍처 문서는 시스템의 주요 결정사항을 명확하게 전달하고, 프로젝트의 목표와 방향성에 대한 공감대를 형성하는 데 중요한 도구임.
- 위험 관리: 초기 아키텍처 설계를 통해 프로젝트의 잠재적 위험을 식별하고 평가할 수 있음. 이는 프로젝트의 실패 가능성을 줄이고, 성공적인 결과를 도출하는 데 기여함.
- 유지보수 및 운영 비용 감소: 명확하고 체계적인 아키텍처는 유지보수 및 운영 단계에서 발생할 수 있는 비용을 크게 줄여 줌. 시스템의 구조와 컴포넌트 간의 상호작용이 명확하면, 문제를 빠르게 진단하고 수정하는 것이 더 용이함.

소프트웨어 아키텍처의 정의

- 안정성 및 가용성 향상: 아키텍처 설계 과정에서 고려된 복원력 및 장애 대비 전략은 시스템의 안정성과 가용성을 높임. 이는 비즈니스 연속성을 보장하고, 사용자 만족도를 높이는 데 기여함.
- 보안 강화: 보안 요구사항을 아키텍처 설계에 통합함으로써, 시스템의 보안을 강화할 수 있음. 초기 단계에서 보안을 고려함으로써, 잠재적인 보안 위협에 대응하는 보다 효과적인 전략을 개발할 수 있음.
- 규모의 경제: 잘 설계된 아키텍처는 재사용 가능한 컴포넌트와 패턴을 통해 규모의 경제를 실현할 수 있게 함. 이는 다른 프로젝트나 애플리케이션에서도 동일한 구성 요소를 재사용함으로써, 개발 비용과 시간을 절약할 수 있게 함.

소프트웨어 아키텍처 스타일

- 소프트웨어 아키텍처 스타일은 시스템의 구조를 정의하는데 사용되는 패턴 또는 규칙의 집합임.
- 각각의 아키텍처 스타일은 특정 문제를 해결하기 위해 고안되었으며, 소프트웨어 시스템의 구성, 컴포넌트 간의 관계, 그리고 데이터 흐름 방식을 지정함.
- 다음은 몇 가지 주요 아키텍처 스타일의 개요임.
 - 계층형 아키텍처 (Layered Architecture)
 - 가장 널리 사용되는 아키텍처 스타일 중 하나로, 시스템을 명확하게 분리된 여러 계층으로 나눔.
 - 각 계층은 특정한 책임을 가지며, 보통 데이터 액세스 계층, 비즈니스 로직 계층, 프레젠테이션 계층 등으로 구성됨.
 - 계층 간의 느슨한 결합을 통해 유지보수성과 개발의 용이성을 증진시킴.

소프트웨어 아키텍처 스타일

- 이벤트 기반 아키텍처 (Event-Driven Architecture, EDA)
 - 시스템 컴포넌트들이 이벤트를 생성하고 반응하는 방식으로 동작하는 아키텍처 스타일임.
 - 이벤트 브로커 또는 메시지 큐를 통해 이벤트를 전달하며, 시스템의 분산과 비동기 처리에 유리함.
 - 실시간 데이터 처리 및 반응형 시스템에 적합함.

소프트웨어 아키텍처 스타일

- 마이크로서비스 아키텍처 (Microservices Architecture)
 - 작고 독립적으로 배포 가능한 서비스들로 구성된 아키텍처 스타일임.
 - 각 마이크로서비스는 특정 비즈니스 기능을 담당하며, API를 통해 서로 통신함.
 - 확장성과 유연성이 뛰어나며, 지속적인 배포 및 개발이 가능함.
- 서비스 지향 아키텍처 (Service-Oriented Architecture, SOA)
 - 재사용 가능한 서비스를 중심으로 구성된 아키텍처 스타일임.
 - 서비스는 네트워크를 통해 액세스되며, 서로 다른 애플리케이션 간의 통합과 상호 작용을 용이하게 함.
 - 비즈니스 프로세스의 유연성과 재사용성을 높이는데 초점을 맞춤.

소프트웨어 아키텍처 스타일

- 클라이언트-서버 아키텍처 (Client-Server Architecture)
 - 클라이언트와 서버 두 가지 주요 컴포넌트로 구성됨.
 - 클라이언트는 서비스를 요청하고, 서버는 이러한 요청을 처리하여 응답함.
 - 웹 어플리케이션과 같은 분산 시스템에 널리 사용됨.
- 모놀리식 아키텍처 (Monolithic Architecture)
 - 시스템의 모든 컴포넌트가 하나의 대규모 소프트웨어 단위로 tightly-coupled 되어 있는 아키텍처 스타일임.
 - 초기 개발과 배포가 간단하며, 작은 규모의 어플리케이션에 적합할 수 있음.
 - 하지만, 어플리케이션이 커짐에 따라 유지보수와 확장성이 어려워질 수 있음.

소프트웨어 아키텍처 스타일

- 클라우드 네이티브 아키텍처 (Cloud-Native Architecture)
 - 클라우드 환경에서 최적으로 동작하도록 설계된 아키텍처 스타일임.
 - 마이크로서비스, 컨테이너, DevOps, CI/CD 등의 기술과 방법론을 활용함.
 - 확장성, 탄력성, 빠른 배포를 가능하게 하며, 클라우드 환경의 이점을 최대한 활용할 수 있음.
- 데이터 중심 아키텍처 (Data-Centric Architecture)
 - 데이터와 데이터 처리에 초점을 맞춘 아키텍처 스타일임.
 - 대규모 데이터 저장소, 빅 데이터 처리 시스템, 데이터 분석 및 처리를 위한 서비스 구조 등을 포함함.
 - 데이터의 저장, 관리, 분석, 시각화를 통해 비즈니스 인사이트를 도출하는 데 중점을 둠.

소프트웨어 아키텍처 스타일

- P2P (Peer-To-Peer) 아키텍처
 - 네트워크 내의 각 노드가 동등한 위치에서 서로 직접 통신하는 분산 네트워크 구조임.
 - 중앙 집중식 서버 없이 데이터 공유, 통신, 분산 컴퓨팅 등을 수행할 수 있음.
 - 파일 공유 시스템, 분산 애플리케이션, 블록체인 기술 등에 활용됨.

소프트웨어 아키텍처 스타일

- 육각형 아키텍처(Hexagonal Architecture), 또는 포트와 어댑터(Ports and Adapters) 아키텍처는 Alistair Cockburn에 의해 제안된 소프트웨어 설계 패턴임.
- 어플리케이션의 핵심 로직을 외부 시스템이나 인터페이스로부터 분리하여, 어플리케이션의 핵심 기능을 여러 입력/출력 채널(예: 사용자 인터페이스, 네트워크, 데이터베이스)과 독립적으로 유지할 수 있도록 하는 것임.
- 육각형 아키텍처(Hexagonal Architecture) 스타일
 - 아키텍처 스타일의 핵심 아이디어는 어플리케이션의 핵심 로직을 외부 시스템이나 인터페이스로부터 분리하여, 어플리케이션의 핵심 기능을 여러 입력/출력 채널(예: 사용자 인터페이스, 네트워크, 데이터베이스)과 독립적으로 유지할 수 있도록 하는 것임

소프트웨어 아키텍처 스타일

- 구조

- 육각형 아키텍처는 크게 세 부분으로 구성됨

- 핵심 도메인: 어플리케이션의 비즈니스 로직을 포함함. 이는 시스템의 핵심 기능을 구현하며, 외부 요소와의 직접적인 상호작용을 하지 않음.
 - 포트(Ports): 핵심 도메인과 외부 세계 사이의 인터페이스 역할을 함. 포트는 시스템으로의 데이터 입력 또는 시스템에서의 데이터 출력을 위한 접점으로 사용됨.
 - 어댑터(Adapters): 외부 시스템이나 기술 스택과 핵심 도메인 사이를 연결하는 구성 요소임. 예를 들어, 데이터베이스, 웹 API, 사용자 인터페이스 등이 어댑터의 역할을 할 수 있음. 어댑터는 포트를 통해 핵심 도메인과 통신함.

소프트웨어 아키텍처 스타일

- 장점

- 분리 및 교체 용이성: 핵심 도메인 로직이 외부 요소로부터 분리되어 있기 때문에, 외부 인터페이스나 데이터 소스를 쉽게 변경하거나 교체할 수 있음.
- 유연성 및 확장성: 새로운 어댑터를 추가함으로써 쉽게 새로운 기능이나 인터페이스를 통합할 수 있으며, 시스템의 확장성이 향상됨.
- 테스트 용이성: 핵심 도메인이 외부 요소로부터 독립되어 있기 때문에, 유닛 테스트와 도메인 로직의 테스트 자동화가 용이함.

소프트웨어 아키텍처 스타일

- 사용 사례

- 육각형 아키텍처는 복잡한 비즈니스 로직과 다양한 입/출력 채널을 가진 시스템에 적합함.
- 예를 들어, 다양한 유형의 클라이언트 인터페이스(웹, 모바일, API 등)를 지원하면서, 핵심 비즈니스 로직을 변경하지 않고 여러 데이터 소스와 통합해야 하는 어플리케이션에 유용함.
- 육각형 아키텍처는 어플리케이션의 유지보수성, 확장성, 테스트 용이성을 향상시키는 동시에, 시스템의 핵심 비즈니스 로직을 외부 요소의 영향으로부터 보호하는 강력한 방법을 제공함.
- 이 아키텍처를 통해 개발자들은 비즈니스 로직에 더 집중할 수 있으며, 기술적인 부분은 어댑터를 통해 유연하게 처리할 수 있음.
- 이러한 분리는 또한 시스템의 각 부분을 독립적으로 개발하고 테스트할 수 있게 해, 전체적인 개발 프로세스의 효율성을 증가시킴.

소프트웨어 아키텍처 스타일

- 육각형 아키텍처는 특히 다음과 같은 상황에서 유리함
 - 다중 채널 인터페이스: 단일 비즈니스 로직을 다양한 인터페이스(웹, 모바일, 데스크탑 등)를 통해 제공해야 할 때, 각 인터페이스에 맞는 어댑터를 구현함으로써 쉽게 확장할 수 있음.
 - 기술 스택의 변화: 사용하는 데이터베이스나 외부 서비스 등의 기술 스택에 변화가 필요한 경우, 해당 부분만의 어댑터를 변경하거나 새로운 어댑터를 추가하여 기술 변경을 용이하게 할 수 있음.
 - 테스트 및 유지보수: 비즈니스 로직이 외부의 구체적인 기술 구현으로부터 분리되어 있기 때문에, 모의 객체(Mock Object)나 가짜 구현을 사용하여 비즈니스 로직의 독립적인 테스트가 가능함. 이는 유닛 테스트와 통합 테스트를 간소화하며, 유지보수를 용이하게 함.

소프트웨어 아키텍처 스타일

- 육각형 아키텍처의 도입은 기존의 모놀리식이나 단순한 계층형 아키텍처를 사용하는 시스템에 비해 초기 설계와 구현이 복잡할 수 있음.
- 장기적으로 보았을 때 시스템의 확장성, 유연성, 테스트 가능성을 크게 향상시키며, 다양한 요구사항과 환경 변화에 효과적으로 대응할 수 있는 견고한 기반을 마련해 줌.
- 육각형 아키텍처는 변화하는 비즈니스 요구사항과 기술 환경에 대응하는 현대적인 소프트웨어 개발 프로젝트에 매우 적합한 아키텍처 스타일로 평가받고 있음.

마이크로서비스 아키텍처는 아키텍처 스타일

- 마이크로서비스 아키텍처는 어플리케이션을 작고 독립적으로 배포 가능한 여러 서비스로 구성하는 아키텍처 스타일임.
- 각 마이크로서비스는 특정 비즈니스 기능을 수행하며, 각각 독립적인 개발, 배포, 확장이 가능함.
- 이러한 구조는 서비스 간의 느슨한 결합을 통해 복잡성을 관리하고, 빠른 개발과 배포를 가능하게 함.
- 마이크로서비스 아키텍처는 대규모 어플리케이션과 시스템에서 특히 유용하며, 지속적인 통합 및 배포(CI/CD), 컨테이너화, 클라우드 컴퓨팅과 같은 현대적인 소프트웨어 개발 및 운영 방법과 잘 어울림.

마이크로서비스 아키텍처는 아키텍처 스타일

- 마이크로서비스 아키텍처의 주요 특징
 - 독립성: 각 마이크로서비스는 독립적인 배포 단위로, 하나의 서비스를 수정하거나 업데이트해도 다른 서비스에 영향을 미치지 않음.
 - 분산 개발: 서비스는 다양한 팀에 의해 독립적으로 개발될 수 있으며, 각 팀은 자신의 서비스에만 집중할 수 있음.
 - 다양한 기술 스택: 서비스별로 가장 적합한 기술 스택을 선택할 수 있어 기술적 유연성이 증가함.
 - 확장성: 시스템의 특정 부분에 대한 수요가 증가할 때, 해당 서비스만을 독립적으로 확장할 수 있음.
 - 내결함성: 한 서비스의 실패가 전체 시스템에 치명적인 영향을 미치지 않도록 설계됨.
 - 자동화된 배포: 마이크로서비스는 컨테이너와 같은 기술을 사용하여 자동화된 환경에서 쉽게 배포하고 관리할 수 있음.

마이크로서비스 아키텍처는 아키텍처 스타일

- 마이크로서비스 아키텍처의 도전 과제
 - 데이터 일관성: 서비스 간 데이터 공유와 일관성 유지가 복잡해질 수 있음.
 - 서비스 간 통신: 서비스가 네트워크를 통해 통신함에 따라 네트워크 지연시간과 데이터 처리량이 성능에 영향을 줄 수 있음.
 - 거대한 시스템: 시간이 지남에 따라 관리해야 할 서비스의 수가 많아질 수 있으며, 이는 시스템의 복잡성을 증가시킴.
 - 보안: 분산된 시스템 특성 상 보안 구현과 관리가 더 복잡해질 수 있음.

마이크로서비스 아키텍처는 아키텍처 스타일

- 마이크로서비스 아키텍처는 이러한 도전 과제를 해결하기 위한 다양한 전략과 도구를 제공함으로써, 현대적인 어플리케이션 개발의 요구사항을 충족시키는 유연하고 확장성 있는 방식을 제함.
- 서비스란 무엇인가?
 - 서비스(Service)는 소프트웨어 아키텍처 분야에서, 특정 기능이나 일련의 작업을 수행하고 그 결과를 제공하는 독립적인 소프트웨어 단위를 의미함.
 - 서비스는 일반적으로 네트워크를 통해 접근 가능하며, 클라이언트-서버 모델에서 클라이언트 애플리케이션이 서비스를 요청하면, 서버 측에서 해당 서비스를 처리하고 결과를 클라이언트에게 반환함.

마이크로서비스 아키텍처는 아키텍처 스타일

- 서비스의 주요 특징
 - 독립성: 서비스는 다른 시스템 구성 요소로부터 독립적으로 운영되며, 독립적인 배포가 가능함.
 - 재사용성: 잘 설계된 서비스는 다양한 애플리케이션에서 재사용될 수 있어, 개발 비용과 시간을 절약할 수 있음.
 - 모듈성: 서비스는 특정 기능을 모듈화하여 제공함으로써, 시스템의 복잡성을 줄이고 관리를 용이하게 함.
 - 표준화된 인터페이스: 서비스는 보통 표준화된 인터페이스(예: REST API)를 통해 접근되므로, 서비스 사용자는 서비스의 내부 구현에 대해 알 필요가 없음.

마이크로서비스 아키텍처는 아키텍처 스타일

- 서비스의 종류
 - 웹 서비스: HTTP를 통해 웹 애플리케이션 간에 상호작용하는 서비스. RESTful API와 SOAP(Simple Object Access Protocol)이 일반적인 웹 서비스 기술임.
 - 마이크로서비스: 소프트웨어 아키텍처 스타일로, 어플리케이션을 작고 독립적으로 배포 가능한 서비스의 집합으로 구성함.
 - 클라우드 서비스: 클라우드 컴퓨팅 환경에서 제공되는 서비스로, 인프라(IaaS), 플랫폼(PaaS), 소프트웨어(SaaS) 등이 있음.

마이크로서비스 아키텍처는 아키텍처 스타일

- 서비스의 활용
 - 서비스는 비즈니스 로직, 데이터 처리, 인증 등 다양한 기능을 제공할 수 있으며, 현대의 소프트웨어 개발에서는 서비스 지향 아키텍처(SOA)나 마이크로서비스 아키텍처 같은 접근 방식을 통해 시스템을 더 유연하고 확장 가능하며, 유지보수하기 쉬운 구조로 설계하는 데 중요한 역할을 함.
 - 서비스 기반의 아키텍처는 조직의 IT 인프라를 최적화하고, 비즈니스 요구사항에 빠르게 대응할 수 있도록 지원함.

마이크로서비스 아키텍처는 아키텍처 스타일

- 느슨한 결합(Loose Coupling)이란 무엇인가?
 - 느슨한 결합(Loose Coupling)은 소프트웨어 설계 원칙 중 하나로, 시스템의 서로 다른 구성 요소들이 최소한의 의존성만을 가지고 상호작용하도록 구성하는 방식을 말함.
 - 각 구성 요소(예: 클래스, 모듈, 서비스)가 독립적으로 개발, 수정, 교체될 수 있도록 하여, 전체 시스템의 유연성, 확장성, 유지보수성을 향상시키는 것을 목표로 함.

마이크로서비스 아키텍처는 아키텍처 스타일

- 느슨한 결합의 주요 특징 및 이점
 - 독립성: 각 구성 요소는 다른 요소들의 구체적인 구현보다는 인터페이스나 계약에 의존함. 이를 통해 한 요소의 변경이 시스템의 다른 부분에 미치는 영향을 최소화할 수 있음.
 - 유연성: 시스템의 한 부분을 변경하거나 교체할 때, 나머지 시스템에 대한 재구성 없이도 쉽게 적용할 수 있음. 이는 새로운 기능의 추가나 기술 스택의 변경을 용이하게 함.
 - 확장성: 느슨한 결합을 통해 시스템의 특정 부분을 독립적으로 확장할 수 있음. 이는 특히 클라우드 환경에서 자원을 효율적으로 사용하는 데 유리함.
 - 재사용성: 잘 정의된 인터페이스를 가진 구성 요소는 다른 시스템이나 애플리케이션에서도 재사용될 가능성이 높아짐.
 - 테스트 용이성: 각 구성 요소가 독립적으로 작동하기 때문에, 단위 테스트가 용이해지고, 테스트 커버리지를 향상시킬 수 있음.

마이크로서비스 아키텍처는 아키텍처 스타일

- 느슨한 결합을 달성하는 방법
 - 인터페이스와 추상화 사용: 구현 대신 인터페이스나 추상 클래스에 의존함으로써, 구성 요소 간의 의존성을 줄일 수 있음.
 - 의존성 주입(Dependency Injection): 구성 요소의 의존성을 외부에서 주입받도록 함으로써, 결합도를 낮출 수 있음.
 - 이벤트 기반 통신: 이벤트 또는 메시지 기반의 비동기 통신을 사용하여, 서로 다른 구성 요소 간의 직접적인 호출을 피함.
 - 모듈화와 서비스 지향 아키텍처(SOA): 시스템을 잘 정의된 모듈이나 서비스로 분리하여 각각의 독립성을 높임.
 - API 계약: 구성 요소 간의 통신을 위한 명확한 API 계약을 정의하여, 각 요소가 예측 가능한 방식으로 상호작용하도록 함.

어플리케이션의 마이크로서비스 아키텍처 정의

- 시스템 작업 식별
- 시스템 운영 정의
- 업종별 분해를 적용하여 서비스 정의 능력 패턴
- 분해 기준을 적용하여 서비스 정의 하위 도메인 패턴

어플리케이션의 마이크로서비스 아키텍처 정의

- 마이크로서비스 아키텍처를 정의하는 방법은 세 단계 과정을 포함하며, 이 과정은 반복적이고 창의적인 접근을 필요로 함.
 - 첫 번째 단계는 애플리케이션의 요구사항을 핵심 요청으로 정제하고, 이를 시스템 작업이라는 추상적 개념으로 표현하는 것임.
 - 두 번째 단계는 서비스로의 분해를 결정하는 것이며, 이는 비즈니스 역량 또는 도메인 주도 설계 서브도메인을 기준으로 할 수 있음.
 - 세 번째 단계에서는 각 서비스의 API를 정의함. 이 과정에서는 각 시스템 작업을 특정 서비스에 할당하고, 필요한 경우 다른 서비스와의 협력 방법을 결정함.

애플리케이션의 마이크로서비스 아키텍처 정의

- 분해 과정에는 여러 장애물이 있을 수 있으며, 이에는 네트워크 지연, 서비스 간 동기 통신에 따른 가용성 감소, 서비스 간 데이터 일관성 유지 필요, 그리고 애플리케이션 전반에 걸쳐 사용되는 소위 신 클래스가 포함됨.
- 장애물을 극복하기 위해 자체 포함된 서비스 개념, 사가, 도메인 주도 설계 개념 등을 사용할 수 있음.
- 이 과정을 통해, 애플리케이션은 기술적 개념이 아닌 비즈니스 개념을 중심으로 조직된 서비스들로 분해될 수 있음.

시스템 작업 식별

- 애플리케이션 아키텍처를 정의하는 첫 단계는 시스템 작업을 정의하는 것임. 이 과정은 애플리케이션의 요구사항, 사용자 스토리, 그리고 이와 관련된 사용자 시나리오로 시작함.
- Craig Larman의 “Applying UML and Patterns”에 영감을 받아, 두 단계 과정을 통해 시스템 작업을 식별하고 정의함.
- 첫 번째 단계에서는 핵심 클래스로 구성된 고수준 도메인 모델을 생성하며, 이 모델은 시스템 작업을 설명하는 어휘를 제공함.
- 두 번째 단계에서는 시스템 작업을 식별하고, 각 작업의 동작을 도메인 모델의 측면에서 설명함.

시스템 작업 식별

- 도메인 모델은 주로 사용자 스토리의 명사에서 유래하고, 시스템 작업은 주로 동사에서 파생됨.
- 도메인 모델은 '이벤트 스토밍' 기법을 사용해서도 정의할 수 있음.
- 각 시스템 작업의 동작은 하나 이상의 도메인 객체와 그들 사이의 관계에 미치는 영향 측면에서 설명되며, 시스템 작업은 도메인 객체를 생성, 업데이트, 삭제할 수 있고, 객체 간의 관계를 생성하거나 파괴할 수도 있음.

시스템 작업 식별

- 상위 수준 도메인 모델 생성
 - 시스템 작업을 정의하는 과정의 첫 단계는 애플리케이션을 위한 고수준 도메인 모델을 스케치하는 것임.
 - 이 모델은 최종적으로 구현될 것보다 훨씬 간단하며, 각 서비스가 자체 도메인 모델을 가지기 때문에 애플리케이션은 단일 도메인 모델을 갖지 않음.
 - 도메인 모델은 이야기와 시나리오의 명사를 분석하고 도메인 전문가와 대화하는 등의 표준 기술을 사용하여 생성됨.
 - 예를 들어, '주문하기' 및 '주문 수락하기' 이야기는 Consumer, Order, Restaurant, CreditCard, Courier, DeliveryInfo 등 다양한 클래스의 존재를 암시함.
 - 최종적으로 몇 차례의 분석 후에 MenuItem, Address와 같은 클래스를 포함하는 도메인 모델이 생성됨.

시스템 작업 식별

- 각 클래스의 책임은 다음과 같음.
 - Consumer: 주문을 하는 소비자.
 - Order: 소비자가 주문한 주문으로, 주문을 설명하고 상태를 추적합니다.
 - OrderLineItem: 주문의 항목.
 - DeliveryInfo: 주문을 배달할 시간과 장소.
 - Restaurant: 소비자에게 배달할 주문을 준비하는 식당.

시스템 작업 식별

- MenuItem: 식당의 메뉴 아이템.
- Courier: 소비자에게 주문을 배달하는 배달원으로, 배달원의 가용성과 현재 위치를 추적합니다.
- Address: 소비자나 식당의 주소.
- Location: 배달원의 위도와 경도.

시스템 작업 정의

- 고수준 도메인 모델을 정의한 후, 다음 단계는 애플리케이션이 처리해야 할 요청들을 식별하는 것임.
- UI의 세부 사항은 각 사용자 시나리오에서 UI가 백엔드 비즈니스 로직에 데이터를 검색하고 업데이트하기 위한 요청을 할 것임을 상상할 수 있음.
- 웹 애플리케이션이기 때문에 대부분의 요청이 HTTP 기반일 것이지만, 일부 클라이언트는 메시징을 사용할 수도 있음.
- 특정 프로토콜에 국한되지 않고 요청을 나타내기 위해 시스템 작업이라는 더 추상적인 개념을 사용하는 것이 합리적임.

업종별 분해를 적용하여 서비스 정의 능력 패턴

- 비즈니스 역량은 비즈니스 아키텍처 모델링에서 유래한 개념으로, 비즈니스가 가치를 생성하기 위해 수행하는 활동을 의미함.
- 주어진 비즈니스의 역량 세트는 비즈니스의 종류에 따라 다름. 예를 들어, 보험 회사의 역량에는 보험 인수, 클레임 관리, 청구, 준수 등이 일반적으로 포함됨.
- 온라인 스토어의 역량에는 주문 관리, 재고 관리, 배송 등이 포함됨.

업종별 분해를 적용하여 서비스 정의 능력 패턴

- 비즈니스 역량은 조직이 하는 일을 정의함.
 - 비즈니스 능력에 따라 마이크로서비스 아키텍처를 분해하는 전략은 비즈니스가 가치를 생성하기 위해 수행하는 작업을 기반으로 함.
 - 비즈니스 아키텍처 모델링에서 가져온 이 개념은, 주어진 비즈니스 유형에 따라 다른 일련의 능력을 가지고 있음을 의미함.
 - 예를 들어, 보험 회사의 경우 주요 비즈니스 능력에는 언더라이팅, 클레임 관리, 청구, 준수 등이 포함될 수 있으며, 온라인 상점의 경우 주문 관리, 재고 관리, 배송 등이 해당됨.

업종별 분해를 적용하여 서비스 정의 능력 패턴

- 비즈니스 역량 식별
 - 조직의 비즈니스 능력은 조직의 비즈니스가 무엇인지를 포착함.
 - 이 능력들은 비즈니스 수행 방식이 시간이 지나며 때로는 극적으로 변할 수 있는 반면, 일반적으로 안정적임.
 - 기술의 빠른 발전으로 많은 비즈니스 프로세스가 자동화됨에 따라 이는 특히 오늘날 더욱 사실임.
 - 예를 들어, 과거에는 은행에서 수표를 입금하기 위해 창구 직원에게 제출했으나, 이후 ATM을 통한 입금이 가능해졌고, 현재는 스마트폰을 사용해 대부분의 수표를 편리하게 입금할 수 있음.
 - 이처럼 '수표 입금'이라는 비즈니스 능력은 안정적으로 유지되었지만, 그 수행 방식은 크게 변화하였음.

업종별 분해를 적용하여 서비스 정의 능력 패턴

- 비즈니스 역량부터 서비스까지
 - 매핑되는 경우가 있으며, 다른 경우에는 하위 능력이 서비스로 매핑됨.
 - 어떤 수준의 능력 계층을 서비스로 매핑할지 결정하는 것은 다소 주관적일 수 있으며, 이에 대한 정당화는 구체적인 상황에 따라 다름.
 - 예를 들어, 공급업체 관리의 하위 능력을 두 가지 서비스로 매핑할 수 있으며, 주문 접수 및 이행 능력은 프로세스의 다른 단계마다 담당하는 세 가지 서비스로 매핑됨.
 - 회계 능력은 유사한 유형의 회계로 인해 자체 서비스로 매핑됨.

업종별 분해를 적용하여 서비스 정의 능력 패턴

- 서비스를 능력을 중심으로 조직화 하는 주요 이점은 능력이 안정적이기 때문에, 결과적으로 얻어지는 아키텍처도 상대적으로 안정적일 것이라는 점임.
- 아키텍처의 개별 구성 요소는 비즈니스의 '어떻게' 측면이 변화함에 따라 진화할 수 있지만, 아키텍처 자체는 변하지 않음.
- 그러나, 이러한 서비스 정의는 어플리케이션 도메인에 대해 더 많이 배우면서 시간이 지남에 따라 진화할 수 있는 초기 시도에 불과함.
- 특히, 서비스 간의 협업 방식을 조사하는 것은 아키텍처 정의 과정에서 중요한 단계임.
- 특정 분해가 과도한 프로세스 간 통신으로 인해 비효율적이라는 것을 발견하거나, 서비스가 복잡성이 증가하여 여러 서비스로 분할할 가치가 있다는 결론에 이를 수 있음.
- 또한, 분해를 방해할 수 있는 여러 장애물이 있어서 처음의 결정을 재고할 수도 있음

분해 기준을 적용하여 서비스 정의 하위 도메인 패턴

- 도메인 모델은 해당 도메인에 대한 지식을 문제 해결에 사용할 수 있는 형태로 포착하며, 팀이 사용하는 어휘, 즉 DDD에서 말하는 공통 언어(Ubiquitous Language)를 정의함.
- 도메인 모델은 어플리케이션의 설계와 구현에 밀접하게 반영됨.
- DDD는 마이크로서비스 아키텍처 적용 시 매우 유용한 두 가지 개념인 서브도메인과 한정된 컨텍스트(bounded contexts)를 제시함.

분해 기준을 적용하여 서비스 정의 하위 도메인 패턴

- 전통적인 엔터프라이즈 모델링 접근법과 달리, DDD는 전체 기업을 위한 단일 모델을 생성하는 대신 각각 명시적 범위를 가진 다수의 도메인 모델을 정의하여, 조직의 다른 부분이 하나의 모델에 동의하는 것의 어려움과 모델의 과도한 복잡성 문제를 피함.
- DDD는 각 서브도메인마다 별도의 도메인 모델을 정의하며, 서브도메인은 어플리케이션 문제 공간의 일부로, 비즈니스 능력 식별 방법과 유사하게 비즈니스를 분석하여 다양한 전문 영역을 식별함으로써 파악됨.

분해 기준을 적용하여 서비스 정의 하위 도메인 패턴

- 한정된 컨텍스트는 도메인 모델을 구현하는 코드 아티팩트를 포함하며, 마이크로서비스 아키텍처 사용 시, 각 한정된 컨텍스트는 하나의 서비스 또는 서비스 세트가 됨.
- DDD를 적용하고 각 서브도메인마다 서비스를 정의함으로써 마이크로서비스 아키텍처를 생성할 수 있음.
- DDD의 서브도메인과 한정된 컨텍스트 개념은 마이크로서비스 아키텍처 내의 서비스에 잘 매핑되며, 마이크로서비스 아키텍처의 자율적 팀 소유 서비스 개념은 DDD의 각 도메인 모델이 단일 팀에 의해 소유되고 개발된다는 개념과 완전히 일치함.
- 자체 도메인 모델을 가진 서브도메인 개념은 갓 클래스를 제거하고 분해를 용이하게 하는 데 훌륭한 방법임.
- "서브도메인에 의한 분해"와 "비즈니스 능력에 의한 분해"는 어플리케이션의 마이크로서비스 아키텍처를 정의하는 두 가지 주요 패턴임.

분해 기준을 적용하여 서비스 정의 하위 도메인 패턴

- 분해 지침
 - 마이크로서비스 아키텍처 패턴을 적용할 때 객체 지향 설계에서의 원칙을 적용하고 사용할 수도 있음.
 - 이 원칙들은 로버트 C. 마틴에 의해 창안되었고, 그의 고전적인 책 " Designing Object Oriented C++ Applications Using The Booch Method " 에 설명되어 있음.
 - 첫 번째 원칙은 클래스의 책임을 정의하는 단일 책임 원칙(Single Responsibility Principle, SRP)이고, 두 번째 원칙은 클래스를 패키지로 조직하는 공통 폐쇄 원칙(Common Closure Principle, CCP)임.

분해 기준을 적용하여 서비스 정의 하위 도메인 패턴

- 단일 책임 원칙
 - 소프트웨어 아키텍처와 설계의 주요 목표 중 하나는 각 소프트웨어 요소의 책임을 결정하는 것임.
 - 단일 책임 원칙(Single Responsibility Principle, SRP)에 따르면, 클래스는 변경되는 하나의 이유만 가져야 함.
 - 클래스가 가진 각 책임은 그 클래스가 변경될 수 있는 잠재적인 이유임. 만약 클래스가 독립적으로 변경되는 다수의 책임을 가지고 있다면, 그 클래스는 안정적이지 않을 것임.
 - SRP를 따름으로써, 각각 단일 책임을 가진 클래스를 정의하게 되어 변경의 이유 또한 단일해짐. SRP를 마이크로서비스 아키텍처 정의에 적용하면, 각각 단일 책임을 가진 작고 응집력 있는 서비스를 생성할 수 있으며, 이는 서비스의 크기를 줄이고 안정성을 높일 것임.

분해 기준을 적용하여 서비스 정의 하위 도메인 패턴

- 공통 폐쇄 원칙
 - 공통 폐쇄 원칙(Common Closure Principle, CCP)은 같은 유형의 변경에 대해 함께 영향을 받는 패키지 내 클래스들이 모여 있어야 한다고 제시함.
 - 이 원칙에 따르면, 동일한 이유로 변경되는 클래스들은 같은 패키지에 속해야 하며, 특정 비즈니스 규칙이 변경될 때 개발자는 제한된 수의 패키지(이상적으로 하나)만 수정하면 됨. 이는 어플리케이션의 유지보수성을 크게 향상시킴.
 - 마이크로서비스 아키텍처 생성 시 CCP를 적용하여 같은 이유로 변경되는 구성 요소들을 동일한 서비스에 포장하면, 요구사항 변경 시 수정 및 배포해야 하는 서비스 수를 최소화할 수 있음.
 - 최적의 경우, 변경사항은 단일 팀과 단일 서비스에만 영향을 미침. CCP는 분산 모놀리스의 안티 패턴에 대한 해결책임.

분해 기준을 적용하여 서비스 정의 하위 도메인 패턴

- 단일 책임 원칙(Single Responsibility Principle, SRP)과 CCP는 로버트 C. 마틴이 개발한 11가지 원칙 중 2가지이며, 마이크로서비스 아키텍처 개발에 특히 유용함.
- 나머지 아홉 가지 원칙은 클래스와 패키지 설계 시 사용됨.
- SRP와 CCP를 포함한 객체 지향 설계(OOD) 원칙에 대한 더 많은 정보는 밥 마틴의 웹사이트에 있는 "The Principles of Object Oriented Design" 기사에서 확인할 수 있음.
- 비즈니스 능력 및 서브도메인에 따른 분해와 SRP, CCP는 어플리케이션을 서비스로 분해하는 좋은 기법임.
- 이를 적용하고 성공적인 마이크로서비스 아키텍처를 개발하기 위해서는, 몇 가지 트랜잭션 관리 및 프로세스 간 통신 문제를 해결해야 함.

애플리케이션을 서비스로 분해하는 데 방해가 되는 장애물

- 비즈니스 능력이나 서브도메인에 해당하는 서비스를 정의함으로써 마이크로서비스 아키텍처를 생성하는 전략은 겉보기에 간단해 보일 수 있음. 그러나, 여러 가지 장애물에 부딪힐 수 있음.
 - 네트워크 지연: 서비스 간 통신은 네트워크를 통해 이루어지기 때문에, 네트워크 지연이 발생할 수 있음. 이는 서비스 응답 시간에 영향을 미쳐 사용자 경험을 저하시킬 수 있음.
 - 동기 통신으로 인한 가용성 감소: 서비스 간에 동기 통신을 사용하면, 한 서비스의 지연이나 실패가 전체 시스템의 가용성에 영향을 미칠 수 있음. 하나의 서비스가 응답을 기다리는 동안 전체 시스템이 멈출 위험이 있음.
 - 서비스 간 데이터 일관성 유지: 마이크로서비스 아키텍처에서 각 서비스는 독립된 데이터 소스를 가질 수 있음. 이는 서비스 간 데이터 일관성을 유지하기 어렵게 만들며, 특히 트랜잭션이 여러 서비스에 걸쳐 있는 경우 더욱 복잡해짐.

애플리케이션을 서비스로 분해하는 데 방해가 되는 장애물

- 데이터의 일관된 뷰 얻기: 서비스 간 데이터가 분산되어 있기 때문에, 시스템 전체에 걸쳐 일관된 데이터 뷰를 제공하는 것이 도전적일 수 있음.
- 갓 클래스로 인한 분해 방해: 특정 클래스가 너무 많은 책임을 가지고 있거나, 시스템의 다양한 기능을 담당하는 경우, 이를 여러 마이크로서비스로 분해하는 것이 어려워질 수 있음.

서비스 API 정의

- 시스템 작업 목록과 잠재적 서비스 목록을 가진 상태에서, 다음 단계는 각 서비스의 API를 정의하는 것임.
- 서비스 API 작업은 외부 클라이언트 또는 다른 서비스에 의해 호출되는 시스템 작업에 해당하거나, 서비스 간 협업을 지원하기 위해 존재함.
- 서비스가 이벤트를 발행하는 주된 목적은 다른 서비스와의 협업을 가능하게 하는 것이며, 이는 서비스 간 데이터 일관성을 유지하는 사가를 구현하거나, 효율적인 쿼리를 지원하는 CQRS 뷰를 업데이트하는 데 사용될 수 있음.
- 또한, 어플리케이션은 외부 클라이언트에게 알림을 주기 위해 이벤트를 사용할 수도 있습니다. 서비스 API를 정의하는 출발점은 각 시스템 작업을 서비스에 매핑하는 것임.
- 그 후, 서비스가 시스템 작업을 구현하기 위해 다른 서비스와 협업할 필요가 있는지 결정하고, 협업을 지원하기 위해 해당 서비스가 제공해야 하는 API를 결정함.

서비스 API 정의

- 서비스에 시스템 작업 할당
 - 요청에 대한 초기 진입점이 되는 서비스를 결정하는 것이 첫 단계임.
 - 많은 시스템 작업은 서비스에 명확하게 매핑 되지만, 때로는 매핑이 덜 명확할 수 있음.
 - 예를 들어, 택배 위치를 업데이트하는 `noteUpdatedLocation()` 작업을 고려해 보면, 이 작업은 택배와 관련되어 있으므로 택배 서비스에 할당되어야 할 것 같지만, 실제로 택배 위치 정보가 필요한 것은 배달 서비스임.
 - 이 경우, 작업에 의해 제공된 정보가 필요한 서비스에 작업을 할당하는 것이 더 나은 선택임.
 - 다른 상황에서는, 작업을 처리하는 데 필요한 정보를 가지고 있는 서비스에 작업을 할당하는 것이 더 의미가 있을 수 있음.

서비스 API 정의

- 서비스 간 협업을 지원하는 데 필요한 API 결정
 - 일부 시스템 작업은 단일 서비스에 의해 전적으로 처리됩니다. 예를 들어, FTGO 어플리케이션에서 `createConsumer()` 작업은 소비자 서비스가 혼자서 완전히 처리함.
 - 그러나 다른 시스템 작업은 여러 서비스에 걸쳐 있음. 이러한 요청을 처리하기 위해 필요한 데이터는 여러 서비스에 흩어져 있을 수 있음.
 - 예를 들어, `createOrder()` 작업을 구현하기 위해서는 주문 서비스가 선행 조건을 검증하고 결과 조건을 실현하기 위해 다음 서비스들을 호출해야 함.
 - 소비자 서비스(소비자가 주문을 할 수 있는지 확인 및 결제 정보 획득), 식당 서비스(주문 항목 검증, 배달 주소/시간이 식당의 서비스 영역 내에 있는지 확인, 최소 주문 요건 충족 및 주문 항목 가격 획득), 주방 서비스(티켓 생성), 회계 서비스(소비자의 신용카드 승인). 마찬가지로, `acceptOrder()` 시스템 작업을 구현하기 위해서는 주방 서비스가 배달 서비스를 호출하여 주문을 배달할 택배를 예약해야 함.

서비스 API 정의

- 서비스, 수정된 API, 그리고 협력자를 보여주는 표를 분석함으로써 서비스 API를 완전히 정의해야 합니다. 지금까지 각 서비스가 구현하는 서비스와 작업을 식별했음.
- 하지만 우리가 개략적으로 그린 아키텍처는 매우 추상적이며, 구체적인 IPC 기술을 선택하지 않았습니다. 용어 "작업"이 동기적인 요청/응답 기반 IPC 메커니즘을 암시하더라도, 비동기 메시징이 중요한 역할을 한다는 것을 알게 될 것임.

마이크로서비스 아키텍처의 프로세스 간 통신

- 마이크로서비스 아키텍처 IPC 개요
- 동기 RPI 패턴
- 비동기 메시징 패턴
- 비동기 메시징을 사용하는 것은 시스템의 가용성을 향상

마이크로서비스 아키텍처 IPC 개요

- 마이크로서비스 아키텍처 IPC 개요
- 상호 작용 스타일
- 마이크로서비스 아키텍처에서 API 정의
- 상호 작용 스타일
- APIs 진화
- 메시지 형식

마이크로서비스 아키텍처 IPC 개요

- 마이크로서비스 아키텍처에서의 프로세스 간 통신(IPC)은 서비스 간의 데이터 및 메시지를 교환하는 방법을 의미함.
- 서비스가 독립적으로 배포되고 실행되기 때문에 중요함.
- IPC 기술은 동기 및 비동기 방식을 포함한 다양한 형태로 존재하며, 각각의 선택은 아키텍처의 성능, 확장성 및 유지 관리에 영향을 미침.
 - 동기 통신
 - 클라이언트가 요청을 보내고 서버로부터 응답을 기다리는 방식임.
 - HTTP/HTTPS를 사용한 RESTful API나 gRPC 같은 프로토콜이 이 범주에 속함.
 - 이 방식은 구현이 간단하고 직관적이지만, 요청 처리 시간 동안 클라이언트가 대기해야 하므로 높은 지연 시간이나 병목 현상을 유발할 수 있음.

마이크로서비스 아키텍처 IPC 개요

- 비동기 통신
 - 요청과 응답이 독립적으로 발생하며, 메시지 큐, 이벤트 버스, 브로커 등을 통해 메시지를 교환함.
 - AMQP, MQTT, Kafka, STOMP 등의 기술이 여기에 해당함.
 - 비동기 통신은 높은 처리량과 확장성을 제공하지만, 메시지 순서, 배달 보장, 상태 관리와 같은 추가적인 고려 사항을 도입함.

마이크로서비스 아키텍처 IPC 개요

- 메시지 형식
 - 데이터 교환 형식으로는 JSON, XML 같은 텍스트 기반 형식과 Protocol Buffers, Avro 같은 이진 형식이 있음.
 - 텍스트 기반 형식은 사람이 읽을 수 있어 디버깅이 용이하지만, 크기가 크고 처리 속도가 느릴 수 있음.
 - 이진 형식은 효율적이고 빠르지만, 사람이 직접 읽기 어렵고, 처리를 위한 추가 도구나 라이브러리가 필요할 수 있음.

상호 작용 스타일

- 마이크로서비스 아키텍처 내의 상호 작용 스타일은 서비스가 어떻게 통신하는지에 대한 다양한 접근 방식을 설명함.
- 마이크로서비스 간의 데이터 교환 방법과 패턴을 정의하며, 각각의 스타일은 특정 상황에 더 적합 할 수 있음. 주요 상호작용 스타일에는 다음과 같은 것들이 있음.
 - 요청/응답(Request/Response)
 - 가장 일반적인 상호작용 스타일로, 한 서비스(클라이언트)가 다른 서비스(서버)에 요청을 보내고, 응답을 기다림. 이 스타일은 동기적이며, HTTP/HTTPS를 사용한 REST API나 gRPC가 이에 해당함.
 - 비동기 메시징(Asynchronous Messaging)
 - 서비스가 메시지 브로커를 통해 메시지를 보내고, 다른 서비스가 나중에 이를 수신함. 이 방식은 서비스 간의 결합도를 낮추고, 확장성을 향상시키며, Kafka, RabbitMQ, AWS SQS와 같은 기술이 사용됨.

상호 작용 스타일

- 이벤트 스트리밍(Event Streaming)
 - 서비스가 발생하는 이벤트를 연속적인 스트림으로 발행하고, 다른 서비스가 이 스트림을 구독하며 이벤트에 반응함. 이는 비동기 메시징의 한 형태로, 대규모 데이터 처리와 실시간 분석에 유용함. Kafka가 대표적인 예.
- RPC(Remote Procedure Call)
 - 클라이언트가 네트워크를 통해 서버의 함수나 프로시저를 마치 로컬에서 호출하듯이 실행할 수 있게 해주는 방식임. gRPC는 이러한 스타일을 구현하는 현대적인 프레임워크 중 하나임.
- 공유 데이터베이스(Shared Database)
 - 모든 서비스가 데이터를 교환하기 위해 같은 데이터베이스에 접근하는 방식임. 이 방식은 데이터 일관성을 유지하기 쉽지만, 서비스 간의 결합도가 높아짐.

상호 작용 스타일

- RESTful API
 - Representational State Transfer의 약자로, 웹 표준을 사용하여 서비스 간의 상호작용을 설계하는 방식임. 이는 상태를 전송하는 대표적인 방법으로, 자원(데이터)의 상태를 HTTP 메소드(GET, POST, PUT, DELETE 등)를 통해 전송함.

마이크로서비스 아키텍처에서 API 정의

- 마이크로서비스 아키텍처에서 API를 정의하는 것은 서비스 간의 통신과 데이터 교환 방법을 명확하게 설정하는 중요한 과정임.
- 서비스가 어떻게 상호작용할지, 어떤 데이터를 주고받을지, 그리고 서비스 경계를 어떻게 정의할지 결정하는 데 도움이 됨.
- 마이크로서비스 아키텍처에서 API를 정의할 때 고려해야 할 주요 원칙과 접근 방식은 다음과 같음.
 - API-먼저 디자인(API-First Design): API-먼저 디자인 접근 방식에서는 API 명세서를 작성하는 것이 소프트웨어 개발 프로세스의 첫 단계임. 이 접근 방식은 API의 일관성과 재사용성을 높이며, 프론트엔드와 백엔드 팀이 독립적으로 작업할 수 있도록 함.
 - 계약 기반 개발(Contract-Based Development): API 계약은 서비스 간의 상호작용을 규정하는 명세서임. 이 계약을 통해 입력, 출력, 데이터 형식, 오류 코드 등 API의 세부 사항이 명확히 정의됨. 계약 기반 개발은 서비스 간의 명확한 경계를 설정하고, 통합 테스트의 기반을 마련함.

마이크로서비스 아키텍처에서 API 정의

- 버전 관리(Versioning): API의 변경 사항을 관리하기 위해 버전 관리가 필수적임. 새로운 기능 추가나 기존 기능 변경 시 기존 클라이언트와의 호환성을 유지하기 위해 새로운 버전의 API를 제공해야 할 수 있음.
- 문서화(Documentation): API 문서화는 개발자가 API를 쉽게 이해하고 사용할 수 있도록 함. 좋은 문서는 API의 엔드포인트, 사용 가능한 메소드, 요청 및 응답 형식 등을 명확하게 설명해야 함.
- 보안(Security): API를 통해 민감한 데이터가 전송될 수 있으므로, 적절한 인증, 인가, 암호화 방법을 적용하는 것이 중요함. OAuth, JWT(Json Web Tokens) 등의 보안 표준을 활용할 수 있음.
- 오류 처리(Error Handling): API는 예상치 못한 오류에 대해 명확한 응답 메시지와 오류 코드를 반환해야 함. 이는 클라이언트가 오류를 쉽게 인식하고 적절히 대응할 수 있도록 함.

마이크로서비스 아키텍처에서 API 정의

- 테스트(Testing): API의 기능성, 성능, 보안 등을 검증하기 위한 충분한 테스트가 필요함. 자동화된 테스트를 포함하여 API가 명세서대로 동작하는지 확인해야 함.

APIs 진화

- API를 진화 시키는 것은 마이크로서비스 아키텍처에서 필수적인 과제임.
- 시간이 지남에 따라 비즈니스 요구사항의 변화, 기술의 발전, 그리고 사용자의 피드백에 따라 서비스를 개선하고 새로운 기능을 추가해야 할 필요가 있음.
- 이 과정에서 API의 변화를 관리하는 방법은 시스템의 안정성과 클라이언트 개발자들의 경험에 큰 영향을 미칩니다. API를 진화 시키는 과정을 관리하는 데 있어 몇 가지 중요한 원칙과 전략이 있음.
 - 버전 관리(Versioning): API의 주요 변경사항을 도입할 때는 새로운 버전을 발행하는 것이 중요합니다. 이는 기존 클라이언트가 변경되지 않은 API를 계속 사용할 수 있게 해주며, 새로운 클라이언트가 최신 기능에 접근할 수 있도록 함. URL, 헤더, 또는 내용 내의 버전 번호를 통해 다른 버전의 API를 구분할 수 있음.
 - 점진적 변경(Incremental Change): 가능한 한 API 변경을 작고 점진적으로 수행하는 것이 좋음. 이 방식은 클라이언트가 새로운 API로의 전환을 더 쉽게 관리할 수 있게 해주며, 시스템의 전반적인 안정성을 유지하는 데 도움이 됨.

APIs 진화

- 사용 중단 정책(Deprecation Policy): 오래되거나 더 이상 사용되지 않는 API 버전을 사용 중단하려고 할 때는 충분한 통지 기간을 제공하는 것이 중요함. 이를 통해 클라이언트 개발자들이 새로운 버전으로 전환할 시간을 가질 수 있음.
- 문서화(Documentation): API의 모든 변경사항은 명확하게 문서화되어야 함. 이는 개발자가 변경사항을 쉽게 이해하고 적절하게 대응할 수 있도록 도와줌.
- 하위 호환성(Backward Compatibility): 새로운 버전의 API를 설계할 때는 가능한 한 이전 버전과의 하위 호환성을 유지하는 것을 목표로 해야 함. 이는 기존 클라이언트가 새 버전의 API를 사용할 때 문제가 없도록 보장함.
- 테스트와 검증(Testing and Validation): API의 변경사항이 기존 기능에 영향을 미치지 않도록 충분한 테스트와 검증이 필요함. 이는 자동화된 테스트를 통해 이루어질 수 있으며, 변경사항이 기존 시스템의 동작을 방해하지 않도록 보장함.

메시지 형식

- 메시지 형식은 마이크로서비스 아키텍처에서 서비스 간 통신에 사용되는 데이터의 구조를 정의함.
- 올바른 메시지 형식을 선택하는 것은 시스템의 성능, 확장성, 그리고 개발자 친화성에 중대한 영향을 미침. 여러 메시지 형식 중에서 가장 널리 사용되는 형식은 다음과 같음.
 - JSON (JavaScript Object Notation): 경량의 데이터 교환 형식으로, 사람이 읽기 쉽고 기계가 파싱하고 생성하기 쉬움. 웹 API와 프론트엔드 애플리케이션 간의 통신에 주로 사용됨.
 - XML (eXtensible Markup Language): 태그를 사용하여 데이터를 구조화하는 또 다른 텍스트 기반 형식임. JSON보다 더 복잡하지만, 네임스페이스와 스키마를 통한 엄격한 데이터 검증 기능을 제공함.
 - YAML (YAML Ain ' t Markup Language): 구성 파일과 메시지 교환에 사용되는 인간 친화적인 데이터 직렬화 형식임. JSON과 유사하지만, 읽기 쉽고 주석을 지원한다는 점에서 차이가 있음.

메시지 형식

- Protocol Buffers: Google에 의해 개발된, 구조화된 데이터를 직렬화하는 데 사용되는 언어 및 플랫폼 독립적인 이진 형식임. 프로토콜 버퍼는 효율적인 데이터 저장 및 통신을 위해 설계되었으며, 크로스 플랫폼 서비스 간 통신에 유용함.
- Avro: 아파치 재단에서 개발한, 리모트 프로시저 호출(RPC)과 데이터 직렬화를 위한 이진 형식임. 스키마를 기반으로 데이터를 직렬화하며, 데이터의 스키마가 메시지와 함께 전송되어 데이터 호환성을 보장함.
- MessagePack: JSON과 유사한 경량의 이진 직렬화 형식임. JSON보다 작고 빠르지만, 여전히 풍부한 데이터 구조를 지원함. 특히 네트워크 대역폭이 제한적인 환경에서 유용함.

동기 RPI 패턴

- 동기적 원격 프로시저 호출(Remote Procedure Invocation, RPI) 패턴은 마이크로서비스 아키텍처에서 서비스 간에 통신하는 데 사용되는 일반적인 방식임.
- 이 패턴은 한 서비스가 다른 서비스의 함수나 메소드를 마치 로컬에서 실행하는 것처럼 호출할 수 있게 해줌.
- RPI는 통신을 요청하고, 그 요청이 처리되어 응답을 받을 때까지 대기하는 동기적 통신 모델을 따름.

동기 RPI 패턴

- 핵심 특징

- 동기성: 클라이언트는 서비스 호출 후 응답을 기다려야 하며, 응답을 받은 후에야 다음 작업을 계속할 수 있음.
- 클라이언트와 서버 간의 상호작용이 간단하고 직관적이지만, 대기 시간에 민감한 작업에서는 성능 저하를 초래할 수 있음.
- 직접적인 메소드 호출: 클라이언트는 원격 서비스의 메소드를 직접 호출하며, 이는 마치 로컬 메소드를 호출하듯이 투명함.
- 블로킹 연산: 동기 RPI는 대개 블로킹 연산을 수행함. 즉, 응답이 도착할 때까지 클라이언트의 실행이 멈춤.

동기 RPI 패턴

- 사용되는 기술
 - HTTP/HTTPS: RESTful API를 통해 구현됨. REST API는 HTTP 메소드(GET, POST, PUT, DELETE 등)를 사용하여 리소스에 대한 CRUD(Create, Read, Update, Delete) 연산을 수행함.
 - gRPC: 구글이 개발한 고성능, 오픈 소스 원격 프로시저 호출 시스템으로, Protocol Buffers를 사용한 효율적인 바이너리 메시징을 기반으로 함.

동기 RPI 패턴

- 고려사항
 - 오류 처리: 네트워크 지연, 타임아웃, 서비스 실패와 같은 문제에 대한 명확한 오류 처리 전략이 필요함.
 - 성능과 확장성: 동기 통신은 응답 대기 시간으로 인해 서비스의 확장성과 전체 시스템의 성능에 영향을 줄 수 있음. 따라서 로드 밸런싱, 타임아웃 설정, 회로 차단기 패턴 등을 고려해야 함.
 - API 버전 관리와 계약: 서비스 계약을 명확히 정의하고, API의 버전 관리를 통해 호환성을 유지하는 것이 중요함.

비동기 메시징 패턴

- 비동기 메시징 패턴은 마이크로서비스 아키텍처에서 서비스 간 통신을 위한 또 다른 핵심적인 방식임.
- 이 패턴은 메시지를 교환함으로써 서비스가 비동기적으로 데이터를 전송하고 수신할 수 있게 함.
- 서비스는 메시지를 보낸 후 응답을 즉시 기다리지 않고 다른 작업을 계속 수행할 수 있음.
- 이 접근 방식은 특히 느슨한 결합, 확장성, 그리고 복잡한 워크플로우 관리에 유용함.

비동기 메시징 패턴

- 핵심 특징

- 비동기성: 메시지 송신자는 수신자가 메시지를 처리할 때까지 기다리지 않음. 이는 시스템 전체의 응답성과 처리량을 향상시킴.
- 느슨한 결합: 송신자와 수신자는 서로에 대한 직접적인 지식 없이 메시지를 교환할 수 있음. 이는 서비스의 독립적인 개발과 배포를 가능하게 함.
- 메시지 큐와 이벤트 스트림: 메시지는 큐에 저장되어 순차적으로 처리되거나, 이벤트 스트림을 통해 실시간으로 분산 처리될 수 있음.
- 확장성: 수신자가 처리할 수 있는 것보다 더 많은 메시지를 보낼 경우, 메시지는 큐에 저장되어 나중에 처리됨. 이는 부하 분산과 시스템 확장성을 지원함.

비동기 메시징 패턴

- 사용되는 기술
 - AMQP (Advanced Message Queuing Protocol): 오픈 스탠다드 메시징 프로토콜로, 큐, 라우팅, 신뢰성 있는 메시지 전달 등을 지원함.
 - Kafka: 고성능, 분산 스트리밍 플랫폼으로, 대규모 메시지 처리와 실시간 데이터 파이프라인 구축에 적합함.
 - RabbitMQ: AMQP를 구현한 오픈 소스 메시지 브로커로, 복잡한 메시징 요구사항과 다양한 메시징 패턴을 지원함.
 - AWS SNS/SQS: AWS에서 제공하는 완전 관리형 메시징 서비스로, 간단하게 메시지를 보내고 큐에서 메시지를 처리할 수 있음.

비동기 메시징 패턴

- 고려사항

- 메시지 처리 보장: 메시지가 한 번만 처리되도록(Exactly Once) 보장하거나, 중복 처리를 허용하는 등의 메시지 전달 보장 수준을 결정해야 함.
- 오류 처리와 복원력: 메시지 처리 실패 시 재시도 로직, 사각지대 없는 모니터링, 오류 대기열 관리 등을 통해 시스템의 복원력을 강화해야 함.
- 메시지 포맷과 직렬화: JSON, XML, Protocol Buffers 등 메시지를 어떻게 직렬화할지 결정하며, 시스템 간 호환성을 고려해야 함.
- 보안: 메시지 데이터의 암호화, 안전한 인증 및 권한 부여 메커니즘을 통해 메시지의 기밀성과 무결성을 보장해야 함.

비동기 메시징 패턴

- 비동기 메시징을 사용하여 시스템의 가용성을 향상시키는 방법은 마이크로서비스 아키텍처의 핵심 전략 중 하나임.
- 비동기 메시징은 서비스 간에 메시지를 교환할 때 호출자가 응답을 즉시 기다리지 않도록 함으로써, 시스템의 병목 현상을 줄이고, 전체적인 응답 시간을 개선할 수 있음.
- 이 접근 방식은 시스템의 가용성과 탄력성을 높이는 데 다음과 같은 이점을 제공함.

비동기 메시징 패턴

- 비동기 메시징의 가용성 향상 이점
 - 탈중앙화된 에러 처리
 - 비동기 메시징 시스템에서는 메시지 처리 실패 시, 실패한 메시지를 재시도 큐에 넣거나, 사건 로그에 기록하는 등의 방법으로 중앙집중식 에러 핸들링에서 벗어날 수 있음.
 - 이는 한 서비스의 실패가 전체 시스템에 영향을 미치는 것을 방지하며, 서비스의 독립적인 가용성을 보장함.
 - 로드 레벨링과 피크 타임 대응
 - 메시지 큐를 통해 과도한 로드를 임시로 저장하고, 시스템의 부하가 줄어들었을 때 처리할 수 있음.
 - 이는 시스템이 피크 타임의 부하를 더 효과적으로 관리할 수 있게 하며, 전체적인 가용성을 향상시킴.

비동기 메시징 패턴

- 분산 시스템의 복원력 증가
 - 비동기 메시징은 서비스 간의 결합도를 낮춤.
 - 이는 시스템의 한 부분에 장애가 발생하더라도, 메시지는 큐에 안전하게 저장되어 있으며, 시스템의 다른 부분은 영향을 받지 않고 동작을 계속할 수 있음을 의미함.
- 동적인 스케일링과 부하 분산
 - 비동기 메시징은 시스템이 동적으로 스케일링되고 부하가 균등하게 분산될 수 있는 기반을 마련함.
 - 메시지 큐 앞에 있는 서비스 인스턴스 수를 늘리거나 줄여서, 수신되는 메시지의 양에 따라 자동으로 처리 용량을 조절할 수 있음.

비동기 메시징 패턴

- 비동기 통신을 통한 효율적 자원 활용
 - 서비스가 요청에 대한 응답을 기다리지 않고 다른 작업을 계속 수행할 수 있기 때문에, 서버 자원을 효율적으로 활용할 수 있음.
 - 이는 시스템의 전반적인 처리량을 증가시키고, 가용성을 높임.

비동기 메시징을 사용하는 것은 시스템의 가용성을 향상

- 비동기 메시징을 사용하는 것은 시스템의 가용성을 향상시키는 효과적인 방법 중 하나임.
- 비동기 메시징 시스템은 메시지를 보내는 측(sender)과 받는 측(receiver) 사이에 직접적인 연결이 없이도 메시지를 교환할 수 있게 해줌.
- 이를 통해 두 시스템 구성 요소 간의 결합도를 낮추고, 각 컴포넌트의 독립성을 높일 수 있음.
 - 비동기 통신: 서비스 A가 서비스 B에 요청을 보낸 후, 응답을 기다리지 않고 다른 작업을 계속 수행할 수 있습니다. 이는 요청 처리 시간 동안 서비스 A가 멈추지 않고 가용 상태를 유지할 수 있음을 의미함.
 - 메시지 큐: 메시지 큐는 발신자와 수신자 사이의 메시지를 임시 저장하는 역할을 함. 수신자가 준비되었을 때 메시지를 처리할 수 있기 때문에, 각 서비스는 자신의 속도에 맞춰 작업을 처리할 수 있으며, 이는 전체 시스템의 가용성을 향상시킴.
 - 부하 분산: 비동기 메시징 시스템은 메시지 처리 부하를 여러 서비스 인스턴스에 분산시킬 수 있음. 이는 고부하 상황에서도 시스템의 가용성을 유지하는 데 도움이 됨.

비동기 메시징을 사용하는 것은 시스템의 가용성을 향상

- 장애 격리: 한 컴포넌트의 실패가 전체 시스템에 영향을 미치는 것을 방지함. 메시지 큐는 시스템의 다른 부분으로 장애가 확산되는 것을 막아주며, 실패한 컴포넌트가 복구된 후에 메시지 처리를 재개할 수 있음.
- 유연한 유지보수와 업그레이드: 비동기 메시징 시스템을 사용하면, 시스템의 일부를 수정하거나 업그레이드하는 동안에도 다른 부분이 계속 작동할 수 있음. 이는 시스템의 가용성을 유지하면서 유지보수 작업을 수행할 수 있게 함.
- 비동기 메시징의 도입 고려 사항
 - 비동기 메시징 시스템을 도입할 때는 몇 가지 중요한 고려 사항이 있음. 메시지의 순서 보장, 데이터 일관성 유지, 오류 처리 및 재시도 메커니즘 구현, 메시지 보안 및 인증 등의 사항을 고려해야 함. 또한, 비동기 메시징 시스템의 모니터링 및 로깅 전략을 수립하여 시스템의 가용성과 성능을 지속적으로 관리할 수 있어야 함.

Agenda

- What is Domain-Driven Design (DDD)?
- Importance of Domain Knowledge
- Strategic Design in Domain-Driven Design(DDD)
- Tactical Design Patterns in Domain-Driven Design (DDD)
- Benefits of Domain-Driven Design(DDD)
- Challenges of Domain-Driven Design (DDD)
- Use-Cases of Domain-Driven Design (DDD)
- Real-world Example of Domain-Driven Design (DDD)

What is Domain-Driven Design(DDD)?

- 도메인 주도 설계(DDD)는 소프트웨어 시스템이 운영되는 문제 도메인을 이해하고 모델링에 중점을 둔 소프트웨어 개발 접근 방식임.
- 도메인 전문가와 긴밀히 협력하여 도메인의 복잡성과 세세한 부분을 깊이 이해하는 것의 중요성을 강조함.
- DDD는 개발자들이 소프트웨어 설계에서 도메인 개념을 효과적으로 포착하고 표현할 수 있도록 돕는 일련의 원칙, 패턴, 실천 방법을 제공함. 방법론이라고 말할 수 있음.

What is Domain-Driven Design(DDD)?

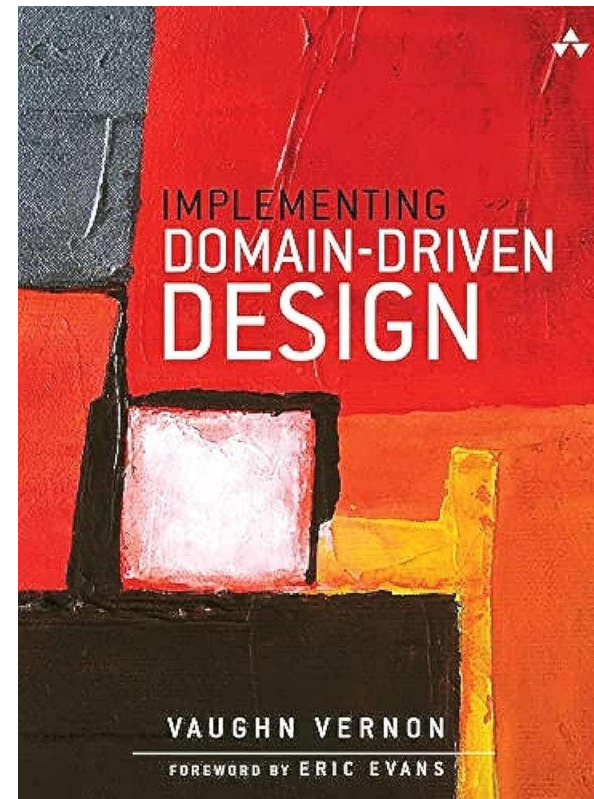
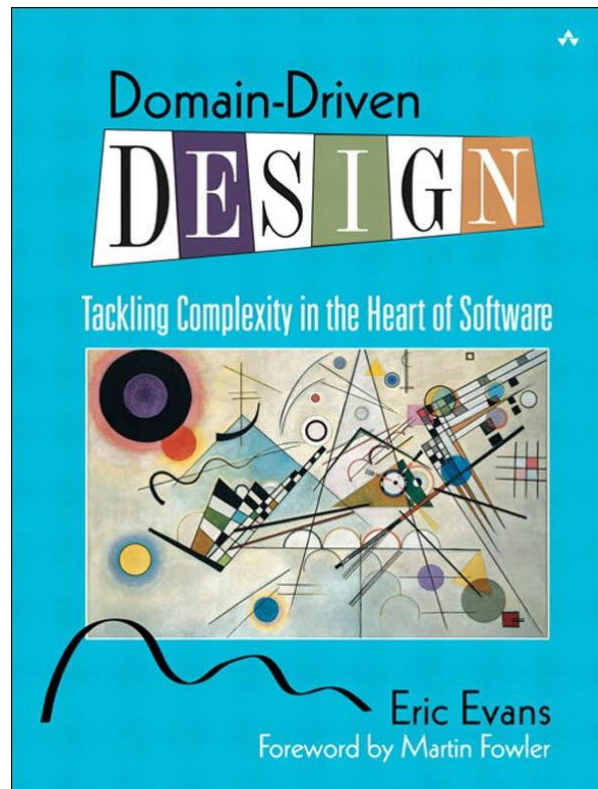
- 도메인(Domain)
 - 소프트웨어 시스템이 해결하려고 하는 주제 영역 또는 문제 공간을 의미함.
 - 소프트웨어가 모델링하거나 지원하려는 실제 세계의 개념, 규칙, 프로세스를 포함함.
 - 예를 들어, 은행 애플리케이션에서 도메인은 계좌, 거래, 고객, 은행 운영과 관련된 규정과 같은 개념을 포함함.
- 주도(Driven)
 - " 주도 " 는 소프트웨어 시스템의 설계가 도메인의 특성과 요구 사항에 의해 안내되거나 영향을 받음을 의미함.
 - 디자인 결정은 기술적 고려사항이나 구현 세부사항에 의해서가 아니라 도메인에 대한 깊은 이해를 바탕으로 함.

What is Domain-Driven Design(DDD)?

- 설계(Design)
 - "설계"는 소프트웨어 시스템에 대한 계획이나 청사진을 만드는 과정을 말함.
 - 시스템이 어떻게 구조화될지, 다양한 구성 요소가 어떻게 상호 작용할지, 그리고 시스템이 기능적 및 비기능적 요구 사항을 어떻게 충족시킬지에 대한 결정을 포함함.
 - 도메인 주도 설계의 맥락에서는 소프트웨어를 도메인의 구조와 행동을 정확하게 반영하는 방식으로 설계하는 데 초점을 맞춤.

What is Domain-Driven Design(DDD)?

- 도메인 주도 설계는 프로그래머 에릭 에반스(Eric Evans)가 2004년에 발표한 그의 책 "도메인 주도 설계: 소프트웨어의 핵심에서 복잡성을 다루기(Domain-Driven Design: Tackling Complexity in Heart of Software)"를 통해 소개된 개념임



Importance of Domain Knowledge

- 최신 기술 스택과 인프라를 사용하여 소프트웨어를 설계하고, 소프트웨어 설계 아키텍처가 놀라울 정도로 훌륭하다고 가정해 본다면,
- 시장에 이 소프트웨어를 출시했을 때, 결국 그 시스템이 훌륭한 지 아닌지를 결정하는 것은 최종 사용자임.
- 시스템이 비즈니스 요구 사항을 해결하지 못한다면, 그것은 누구에게도 쓸모가 없음. 시스템이 얼마나 예쁘게 보이든, 아키텍처의 인프라가 얼마나 잘 구성되어 있든 상관없음.
- 에릭 에반스(Eric Evans)에 따르면, "우리가 소프트웨어를 개발할 때 초점은 기술이 아니라 주로 비즈니스에 맞춰져야 합니다. 기억하세요".
- "고객이 무엇을 원하는지 알아야 할 책임은 고객에게 있는 것이 아니다" - 스티브 잡스

Importance of Domain Knowledge

- 소프트웨어 개발과 제품 디자인에서 근본적인 진실을 반영함. 시스템의 성공 여부는 그것이 최종 사용자에게 실제로 관련성이 있고 유용하며 특정 비즈니스 요구 사항을 어떻게 해결하는지에 의해 결정됨.
- 이 관점은 도메인 주도 설계(DDD)와 사용자 중심 디자인 철학의 원칙과 밀접하게 일치함.
- 에릭 에반스는 도메인 주도 설계를 옹호하며, 소프트웨어가 개발되는 도메인(또는 비즈니스)을 깊이 이해하는 것의 중요성을 강조함.
- 초점은 기술을 목적 자체로 보는 것이 아니라, 소프트웨어를 통해 비즈니스 문제를 해결하는 수단으로 보는 것임.
- 이러한 접근 방식은 소프트웨어가 사용자에게 실제 가치를 제공하고 비즈니스의 전략적 목표를 지원한다는 것을 보장함.

Importance of Domain Knowledge

- 스티브 잡스의 말, "고객이 무엇을 원하는지 알아야 할 책임은 고객에게 있는 것이 아니다",는 사용자의 요구를 예측하고, 종종 사용자들이 그들의 요구를 명확하게 표현하기 전에 그것을 인식하는 제품 개발자와 혁신가의 역할을 강조함. 이는 사용자의 맥락, 도전 과제, 목표를 깊이 이해하고, 지속적인 공감과 관찰에 대한 약속을 요구함.

Importance of Domain Knowledge

- 이러한 관점을 결합하면, 성공적인 소프트웨어 개발을 위한 몇 가지 핵심 원칙이 도출됨.
 - 사용자 중심 디자인: 사용자를 먼저 생각함. 그들의 문제, 워크플로우, 목표를 이해하십시오. 사용자의 요구를 직접적으로 해결하는 직관적인 솔루션을 설계함.
 - 비즈니스 정렬: 소프트웨어가 사용자의 요구뿐만 아니라 더 넓은 비즈니스 목표와 전략과도 일치하는지 확인함. 이러한 정렬은 소프트웨어가 비즈니스의 성공에 의미 있는 방식으로 기여하도록 보장하는 데 도움이 됨.
 - 혁신적인 접근: 사용자나 시장이 직접 요구하지 않는 해결책을 제시함으로써 혁신을 주도함. 이는 깊은 시장 및 사용자 이해를 바탕으로, 기존에 인식되지 않았던 요구를 충족시킬 수 있는 새로운 기회를 탐색하는 것을 의미함.

Importance of Domain Knowledge

- 지속적인 학습과 적응: 시장, 기술, 사용자 요구는 지속적으로 변화함. 성공적인 소프트웨어 개발은 이러한 변화를 인식하고, 학습하며, 이에 적응하는 능력을 필요로 함. 이는 지속적인 사용자 피드백 수집과 반응, 그리고 비즈니스 목표와 전략의 변화에 대응하는 민첩한 개발 접근 방식을 통해 달성될 수 있음.
- 팀 간의 소통 강화: 개발자, 디자이너, 비즈니스 분석가, 사용자 경험 전문가 등 프로젝트에 참여하는 모든 팀원 간의 효과적인 소통과 협업은 성공적인 소프트웨어 개발의 핵심임. 모든 팀원이 공동의 목표와 비전을 공유하고, 상호 이해를 바탕으로 협력할 때, 사용자와 비즈니스 모두에게 가치를 제공하는 솔루션을 만들 수 있음.

Strategic Design in Domain-Driven Design(DDD)

- 도메인 주도 설계(DDD)에서의 전략적 설계는 문제 도메인과 일치하는 방식으로 소프트웨어 시스템의 전체 아키텍처와 구조를 정의하는 데 중점을 둠.
- 도메인 개념을 어떻게 조직할지, 시스템을 관리 가능한 부분으로 어떻게 분할할지, 그리고 다른 구성 요소 간에 명확한 경계를 어떻게 설정할 지와 같은 고차원적인 문제를 다룸.
- 도메인 주도 설계(DDD) 내의 전략적 설계의 주요 개념에는 다음이 포함됨
 - 경계 컨텍스트(Bounded Contexts): 특정 모델이나 언어가 일관되게 적용되는 전체 문제 도메인 내의 특정 영역을 나타냄. 시스템의 다른 부분들은 같은 용어에 대해 다른 의미를 가질 수 있으며, 경계 컨텍스트는 이러한 용어가 특정 의미를 가지는 명시적 경계를 정의함.
 - 컨텍스트 매핑(Context Mapping): 다른 경계 컨텍스트들 사이의 관계와 상호작용을 정의하는 과정임. 이는 컨텍스트들 사이의 중첩 영역이나 통합을 식별하고 그 사이의 커뮤니케이션 채널과 합의를 설정하는 것을 포함함.

Strategic Design in Domain-Driven Design(DDD)

- 전략적 패턴(Strategic Patterns): 문제 도메인과 일치하는 방식으로 소프트웨어 시스템의 아키텍처를 조직화하기 위한 일반적인 지침이나 원칙임. 이러한 패턴은 복잡한 시스템을 설계할 때 흔히 마주치는 도전을 해결하고, 시스템을 효과적으로 구조화하기 위한 검증된 접근 방식을 제공함.
- 공유 커널(Shared Kernel): 다른 경계 컨텍스트들 사이에 공통성을 식별하고 여러 컨텍스트가 사용하는 도메인 모델의 공유된 하위 집합을 설정하는 전략적 패턴임.
- 반부패 계층(ACL:Anti-Corruption Layer): 외부 또는 레거시 시스템의 영향으로부터 시스템을 보호하는 전략적 패턴임. ACL은 외부 시스템과 코어 도메인 모델 사이의 번역 계층으로 작용하여 데이터와 메시지를 필요에 따라 변환함.
- 유비쿼터스 언어(Ubiquitous Language): 소프트웨어 시스템 개발에 관련된 모든 이해관계자가 일관되게 사용하는 공유 어휘나 언어를 의미함. 이 언어는 도메인 지식과 개념을 정확하게 대표하는 용어, 구절, 개념으로 구성됨.

Strategic Design in Domain-Driven Design(DDD)

- 유비쿼터스 언어의 주요 원칙은 다음과 같음.
 - 공유된 이해(Shared Understanding): 개발 팀 전체가 문제 도메인에 대한 공유된 이해를 확립하는 것이 유비쿼터스 언어의 주요 목표임..
 - 일관성과 명확성 (Consistency & Clarity): 유비쿼터스 언어는 정확하고 모호하지 않은 용어를 사용함으로써 의사소통에서 일관성과 명확성을 촉진함. 언어 내의 각 용어나 구문은 명확하고 합의된 의미를 가져야 함.
 - 비즈니스 개념과의 일치 (Alignment with Business Concepts): DDD에서 사용되는 언어는 비즈니스 도메인에서 사용되는 용어와 개념과 밀접하게 일치해야 함. 이는 도메인 전문가가 문제 도메인에 대해 생각하고 말하는 방식을 반영해야 하며, 소프트웨어가 현실 세계의 개념과 프로세스를 정확하게 대표하도록 보장함.

Strategic Design in Domain-Driven Design(DDD)

- 진화하는 성격(Evolutionary Nature): 유비쿼터스 언어는 정적이지 않고 시간이 지남에 따라 변화함. 팀이 도메인에 대한 더 깊은 이해를 얻고, 요구 사항이 변경됨에 따라, 언어는 새로운 통찰력, 발견 또는 비즈니스 우선순위의 변화를 반영하도록 조정되어야 함. 이는 개발 과정 내내 언어가 관련성을 유지하고 최신 상태를 유지하도록 보장함.

Tactical Design Patterns in Domain-Driven Design (DDD)

- 도메인 주도 설계(DDD)의 전략적 설계는 복잡한 시스템을 효과적으로 설계하고 구현하는 데 필수적인 개념, 패턴, 그리고 실천 방법을 제공함.
- 경계 컨텍스트와 컨텍스트 매핑을 통해 시스템의 다양한 부분 사이의 명확한 경계를 정의하고, 전략적 패턴을 활용하여 시스템 아키텍처를 문제 도메인과 일치시키며, 유비쿼터스 언어를 통해 모든 이해관계자 간의 효과적인 의사소통과 공유된 이해를 촉진함.
- 이러한 접근 방식은 도메인의 복잡성을 관리하고, 소프트웨어가 실제 비즈니스 요구사항을 정확하게 반영할 수 있도록 지원함.
- 도메인 주도 설계(DDD)에서의 기술적 설계 패턴은 소프트웨어 시스템 내 도메인 모델을 구조화하고 조직화 하는 특정 전략이나 기술임.
- 이러한 패턴은 개발자가 도메인의 복잡성을 효과적으로 포착하고 유지보수성, 유연성, 확장성을 증진시키는 데 도움을 줌.

Tactical Design Patterns in Domain-Driven Design (DDD)

- DDD에서 주요 기술적 설계 패턴은 다음과 같음.
 - 엔티티(Entity)
 - 엔티티는 독특한 신원과 생명주기를 가진 도메인 객체임. 엔티티는 고유 식별자와 변경 가능한 상태로 특징지어지며, 도메인 내 특정 개념과 관련된 행동과 데이터를 캡슐화함.
 - 예를 들어, 은행 애플리케이션에서는 계좌번호, 잔액, 소유자와 같은 속성과 함께 입금, 출금, 이체 방법을 가진 '은행 계좌' 엔티티가 있을 수 있음.

Tactical Design Patterns in Domain-Driven Design (DDD)

- 값 객체(Value Object)
 - 값 객체는 개념적으로 변경 불가능한 값을 나타내는 도메인 객체임. 엔티티와 달리 값 객체는 독특한 신원을 가지지 않으며, 일반적으로 엔티티의 속성이나 특성을 나타내는 데 사용됨. 값 객체는 신원이 아닌 속성을 기반으로 동등성 비교가 가능함.
 - 예를 들어, '돈' 값 객체는 통화 유형과 금액과 같은 속성을 캡슐화 하여 특정 금액의 통화를 나타낼 수 있음.

Tactical Design Patterns in Domain-Driven Design (DDD)

- 집합체(Aggregate)
 - 집합체는 데이터 일관성과 거래적 무결성을 위해 단일 단위로 취급되는 도메인 객체의 집합임. 집합체는 하나 이상의 엔티티와 값 객체로 구성되며, 한 엔티티가 집합체 루트로 지정됨.
 - 예를 들어, 전자 상거래 시스템에서 '주문' 집합체는 '주문 항목'과 '고객'과 같은 엔티티로 구성될 수 있으며, '주문' 엔티티가 집합체 루트 역할을 할 수 있음.
- 리포지토리(Repository)
 - 리포지토리는 도메인 모델에서 데이터 접근과 지속성 로직을 추상화하는 메커니즘임. 리포지토리는 도메인 객체를 쿼리하고 저장하기 위한 표준화된 인터페이스를 제공하며, 데이터가 검색되거나 저장되는 방법의 세부사항을 숨김.
 - 예를 들어, '고객 리포지토리'는 고객 엔티티를 검색하고 저장하는 방법을 제공할 수 있음.

Tactical Design Patterns in Domain-Driven Design (DDD)

- 팩토리(Factory)

- 팩토리는 복잡한 도메인 객체의 인스턴스를 생성하기 위한 로직을 캡슐화하는 생성 패턴임. 팩토리는 객체의 생성 과정을 추상화하여, 클라이언트가 객체의 구체적인 구성 세부사항을 알 필요 없이 객체를 생성할 수 있게 함. 복잡한 초기화 로직이 필요하거나 여러 단계를 거쳐야 하는 객체를 생성할 때 특히 유용함.
- 예를 들어, '제품 팩토리'는 기본 구성으로 '제품' 엔티티의 인스턴스를 생성하는 책임을 질 수 있음.

- 서비스(Service)

- 서비스는 특정 엔티티나 값 객체에 자연스럽게 속하지 않는 행동이나 연산을 나타내는 도메인 객체임. 서비스는 여러 객체를 대상으로 하는 도메인 로직을 캡슐화 하거나 객체 간의 상호작용을 조정함. 일반적으로 서비스는 상태가 없으며 특정 작업 수행이나 도메인 규칙 집행에 집중함.
- 예를 들어, '주문 서비스'는 주문 처리, 할인 적용, 배송비 계산과 같은 메서드를 제공할 수 있음.

Benefits of Domain-Driven Design(DDD)

- 도메인 주도 설계(DDD)는 다음과 같은 핵심 원칙에 중점을 둠.
 - 공유된 이해(Shared Understanding): 유비쿼터스 언어를 통해 도메인 전문가, 개발자, 이해관계자 간의 협력을 장려하고, 팀이 더 효과적으로 의사소통하며, 소프트웨어가 비즈니스의 필요와 요구 사항을 정확하게 반영하도록 함.
 - 핵심 도메인에 대한 초점(Focus on Core Domain): 팀이 애플리케이션의 핵심 도메인을 식별하고 우선순위를 두어, 비즈니스에 가장 큰 가치를 제공하는 시스템 영역에 개발 노력을 집중함.
 - 변화에 대한 탄력성(Resilience to Change): 도메인을 모델링함으로써 문제 도메인의 내재된 복잡성과 불확실성을 반영하는, 변화에 탄력적인 소프트웨어 시스템을 설계함.
 - 관심사의 명확한 분리(Clear Separation of Concerns): 도메인 로직, 인프라스트럭처, 사용자 인터페이스 사이에 명확한 분리를 장려하여, 도메인 로직을 기술적 세부 사항 및 인프라스트럭처 관심사로부터 격리함.

Benefits of Domain-Driven Design(DDD)

- 향상된 테스트 가능성(Improved Testability): 잘 정의된 경계와 행동을 가진 도메인 객체의 사용을 촉진하여, 도메인 로직의 정확성을 검증하는 더 나은 테스트 작성을 용이하게 함.
- 복잡한 비즈니스 규칙 지원(Support for Complex Business Rules): 도메인 모델 내에서 복잡한 비즈니스 규칙과 워크플로우를 모델링하고 구현하기 위한 패턴과 기술을 제공함.
- 비즈니스 목표와의 정렬(Alignment with Business Goals): 소프트웨어 개발 노력을 비즈니스의 전략적 목표와 목적에 맞추며, 문제 도메인을 이해하고 모델링함으로써 비즈니스 목표를 직접 지원하는 소프트웨어 솔루션을 제공함.

Challenges of Domain-Driven Design (DDD)

- 도메인 주도 설계(DDD)를 도입하며 마주칠 수 있는 주요 도전 과제는 다음과 같음.
 - 복잡성(Complexity): 크고 복잡한 도메인에서 DDD는 특히 복잡성을 증가시킬 수 있음. 복잡한 비즈니스 도메인을 정확하게 모델링하려면 도메인에 대한 깊은 이해가 필요하며, 모호성과 불확실성을 다루는 일이 포함될 수 있음.
 - 유비쿼터스 언어 도입(Ubiquitous Language Adoption): 개발자와 도메인 전문가 간 협력을 통해 도메인 개념을 정확하게 대표하는 공유 어휘를 확립하고 유지하는 것이 도전적일 수 있음. 유비쿼터스 언어에 대한 합의를 이루기 위해서는 의사소통 장벽을 극복하고 용어 및 관점의 차이를 조율해야 할 수 있음.
 - 경계 컨텍스트 정렬(Bounded Context Alignment): 다른 부분들이 서로 다른 모델과 경계 컨텍스트를 가질 수 있는 크고 복잡한 도메인에서, 이들을 정렬하고 일관성을 유지하는 것은 어려울 수 있습니다. 이는 팀 간 명확한 의사소통, 협업, 조정을 요구함.

Challenges of Domain-Driven Design (DDD)

- 기술적 복잡성(Technical Complexity): DDD 원칙과 패턴을 효과적으로 구현하기 위해 새로운 기술, 프레임워크, 아키텍처 접근 방식을 채택할 필요가 있을 수 있음. DDD를 기존 시스템이나 레거시 코드베이스와 통합하는 것은 복잡할 수 있으며, DDD 원칙에 맞게 기존 코드를 리팩토링하거나 재설계해야 할 수 있음.
- 변화에 대한 저항(Resistance to Change): 전통적인 개발 접근 방식에 익숙한 팀원들 사이에서 DDD 도입은 저항을 만날 수 있으며, DDD를 지나치게 복잡하거나 비실용적으로 여길 수 있음.
- 과도한 설계(Over-Engineering): DDD를 적용하면서 복잡한 도메인 개념을 모델링하는 데 너무 집중하고 불필요한 추상화나 복잡성을 도입하는 과도한 설계의 위험이 있음. 설계와 구현을 과도하게 복잡하게 만들지 않기 위해 간결성과 표현력 사이의 적절한 균형을 찾는 것이 중요함.

Use-Cases of Domain-Driven Design (DDD)

- 도메인 주도 설계(DDD)는 다양한 분야에서 복잡한 도메인 개념과 프로세스를 모델링하는 데 효과적임.
 - 금융 및 은행업: DDD는 복잡한 금융 도구, 거래, 규제 요구사항을 모델링하는 데 사용될 수 있으며, 계좌, 거래, 포트폴리오와 같은 도메인 개념을 정확하게 표현함으로써 금융 시스템의 무결성과 일관성을 보장합니다. 이는 리스크 관리, 준수 및 보고의 개선을 가능하게 함.
 - 전자 상거래 및 소매업: 전자 상거래 플랫폼은 제품 카탈로그, 재고 관리, 가격 책정, 고객 주문 등 복잡한 도메인 개념을 다룸. DDD는 이러한 개념을 효과적으로 모델링하여, 개인화된 추천, 동적 가격 책정, 간소화된 주문 처리와 같은 기능을 가능하게 함.
 - 헬스케어 및 생명 과학: 헬스케어에서 DDD는 환자 기록, 의료 진단, 치료 계획, 헬스케어 워크플로우를 모델링하는 데 사용될 수 있으며, 환자 인구 통계, 의료 기록, 임상 프로토콜과 같은 도메인 개념을 정확하게 표현하여, 강력한 전자 건강 기록(HER) 시스템, 의료 영상 플랫폼, 원격 의료 애플리케이션의 개발을 가능하게 함.

Use-Cases of Domain-Driven Design (DDD)

- 보험: 보험사는 다양한 제품, 정책, 클레임, 인수 과정을 관리함. DDD는 이러한 복잡한 도메인 개념을 모델링하는 데 도움을 주어, 정책 관리, 클레임 처리, 리스크 평가, 보험 계리 분석과 같은 기능을 가능하게 함.
- 부동산 및 재산 관리: 부동산 및 재산 관리는 다양한 속성, 임대 계약, 세입자, 유지 보수 요청, 재정 거래를 처리함. DDD는 이러한 도메인 개념을 효과적으로 모델링하여, 부동산 목록, 임대 관리, 세입자 포털, 자산 추적과 같은 기능을 가능하게 함.

Real-world Example of Domain-Driven Design (DDD)

- Problem Statement
 - "RideX"라는 차량 공유 애플리케이션을 개발한다고 가정해봅시다. 이 시스템은 사용자가 차량을 요청하고, 운전자가 차량 요청을 수락하며, 사용자와 운전자 간의 차량을 조정할 수 있게 합니다.
- Ubiquitous Language
 - 사용자: RideX 플랫폼을 통해 차량을 요청하는 개인을 의미합니다.
 - 운전자: RideX 플랫폼을 통해 사용자에게 차량을 제공하는 개인을 의미합니다.
 - 차량 요청: 사용자의 차량 요청을 나타내며, 픽업 위치, 목적지, 차량 선호도와 같은 세부사항을 지정합니다.
 - 차량: 픽업 및 하차 위치, 요금, 소요 시간 등의 세부사항을 포함하는 단일 차량 인스턴스를 나타냅니다.
 - 차량 상태: "요청됨", "수락됨", "진행 중", "완료됨"과 같은 차량의 현재 상태를 나타냅니다.

Real-world Example of Domain-Driven Design (DDD)

- Bounded Contexts
 - 차량 관리 컨텍스트: 차량 요청, 운전자에게 차량 할당, 차량 상태 업데이트를 포함한 차량의 생명주기 관리를 담당합니다.
 - 사용자 관리 컨텍스트: 사용자 인증, 등록 및 차량 이력, 결제 방법과 같은 사용자 특정 기능을 처리합니다.
 - 운전자 관리 컨텍스트: 운전자 인증, 등록, 가용 상태 및 수익, 평점과 같은 운전자 특정 기능을 관리합니다.

Real-world Example of Domain-Driven Design (DDD)

- Entites & Value Objects
 - 사용자 엔티티: 사용자 ID, 이메일, 비밀번호, 결제 정보와 같은 속성을 가진 RideX 플랫폼의 등록된 사용자를 나타냅니다.
 - 운전자 엔티티: 운전자 ID, 차량 세부사항, 운전자 상태와 같은 속성을 가진 RideX 플랫폼의 등록된 운전자를 나타냅니다.
 - 차량 요청 엔티티: 요청 ID, 픽업 위치, 목적지, 차량 선호도와 같은 속성을 포함하는 사용자의 차량 요청을 나타냅니다.
 - 차량 엔티티: 차량 ID, 픽업 및 하차 위치, 요금, 차량 상태와 같은 속성을 포함하는 단일 차량 인스턴스를 나타냅니다.
 - 위치 값 객체: 위도와 경도와 같은 속성을 가진 지리적 위치를 나타냅니다.

Real-world Example of Domain-Driven Design (DDD)

- Aggregates
 - 차량 집합체: 차량 요청, 사용자, 운전자 엔티티와 같은 관련 엔티티를 포함하며, 차량 엔티티를 집합체 루트로 가지고 있습니다. 차량 집합체는 차량 요청 처리, 운전자 할당, 차량 상태 업데이트와 같은 차량의 생명주기 관리 로직을 캡슐화합니다.
- Repository
 - 차량 리포지토리: 차량 세부사항 검색, 차량 상태 업데이트, 데이터베이스에 차량 관련 데이터 저장과 같은 차량 관련 엔티티를 쿼리하고 저장하는 메서드를 제공합니다.

Real-world Example of Domain-Driven Design (DDD)

- Services
 - 차량 할당 서비스: 운전자 가용성, 픽업 위치와의 근접성, 사용자 선호도와 같은 요소를 기반으로 사용자의 차량 요청에 대해 가용한 운전자를 할당하는 역할을 담당합니다.
 - 결제 서비스: 완료된 차량의 결제 처리를 관리하며, 요금을 계산하고 결제를 처리하고 사용자 및 운전자의 결제 정보를 업데이트합니다.

Real-world Example of Domain-Driven Design (DDD)

- Domain Events
 - 차량 요청 이벤트(RideRequestedEvent): 사용자가 차량을 요청할 때 발생하는 이벤트로, 차량 요청 세부사항과 사용자 ID와 같은 정보를 포함합니다.
 - 차량 수락 이벤트(RideAcceptedEvent): 운전자가 차량 요청을 수락할 때 발생하는 이벤트로, 차량 ID, 운전자 ID, 픽업 위치와 같은 정보를 포함합니다.

Real-world Example of Domain-Driven Design (DDD)

- 예)
 - 사용자가 차량을 요청함: 사용자는 픽업 위치, 목적지 및 차량 선호도를 제공하여 차량을 요청합니다. RideX는 새로운 차량 요청 엔티티를 생성하고 차량요청이벤트를 트리거합니다.
 - 운전자가 차량을 수락함: 운전자가 RideX 플랫폼에서 차량 요청을 수락합니다. RideX는 차량 상태를 "수락됨"으로 업데이트하고, 운전자를 차량에 할당한 다음, 차량 수락이벤트를 트리거 합니다.
 - 차량 진행 중: 사용자와 운전자는 차량을 조율하며, 운전자가 픽업 위치에 도달하면 차량 상태는 "진행 중"으로 전환됩니다.
 - 차량 완료: 목적지에 도착한 후, 차량 상태는 "완료됨"으로 업데이트됩니다. RideX는 요금을 계산하고, 결제를 처리하며, 사용자 및 운전자의 결제 정보를 그에 따라 업데이트합니다.